

Model Compression and Efficient AI

Interview-focused guide for ML placements, AI engineer roles, research internships, and practical projects. Each chapter is self-contained; examples use PyTorch and common deployment libraries.

Quantization

1. Overview

Quantization represents model weights and/or activations with fewer bits. A model trained in `float32` may be executed with `float16`, `bfloat16`, `int8`, or even 4-bit values. This reduces model size, memory bandwidth, energy use, and often latency. It is widely used in mobile inference, CPU serving, TensorRT deployments, and LLM serving.

Quantization does not automatically make every model faster. Speedup requires kernels and hardware that efficiently support the chosen dtype; conversion overhead or unsupported operators can erase the benefit.

2. Intuition

Suppose weights range from `-1.0` to `1.0`. Instead of storing every decimal precisely, map the range to 256 integer levels. Nearby values share the same bucket, so a little precision is lost, but storage falls from 32 bits to 8 bits per value. It resembles saving an image with a smaller color palette.

3. Prerequisites

- Floating-point and integer representations
- Tensors, neural-network inference, calibration data
- Min/max, clipping, rounding, mean squared error
- PyTorch modules and model evaluation
- Hardware awareness: CPU vector instructions, GPU tensor cores, accelerators

4. Core Concepts

Subtopic	Meaning	Why it matters	Simple example	Common interview angle
Post-training quantization (PTQ)	Quantize an already-trained model	Fast; no full retraining	FP32 CNN to INT8 using 500 calibration images	PTQ vs QAT
Quantization-aware training (QAT)	Simulate quantization during training	Usually preserves more accuracy	Fine-tune with fake-quant nodes	Why fake quantization is used

Subtopic	Meaning	Why it matters	Simple example	Common interview angle
Dynamic quantization	Quantize weights ahead of time and activations at runtime	Simple for CPU linear/RNN models	INT8 <code>nn.Linear</code>	Dynamic vs static
Static quantization	Calibrate fixed activation ranges before inference	Faster inference, more preparation	INT8 CNN	Role of calibration
Symmetric quantization	Zero maps to zero; usually one scale	Efficient arithmetic	Range <code>[-a, a]</code>	Why weights are often symmetric
Asymmetric quantization	Uses scale and nonzero zero-point	Better for skewed/nonnegative activations	ReLU range <code>[0, 6]</code>	Zero-point derivation
Per-tensor	One scale for a tensor	Cheap metadata	One scale for a layer	Accuracy trade-off
Per-channel	Separate scale per output channel	Better for varying weight distributions	Scale each convolution filter	Why common for weights
Mixed precision	Different dtypes for different operations	Protect sensitive layers	INT8 matmuls, FP16 softmax	Which operations stay high precision

5. Algorithm / Working Process

For static PTQ:

1. Train and evaluate an FP32 model to establish a baseline.
2. Fuse eligible operations such as convolution, batch normalization, and ReLU.
3. Insert observers to record activation distributions.
4. Run representative calibration samples through the model.
5. Choose scales, zero-points, granularity, and clipping ranges.
6. Replace observed operations with quantized kernels.
7. Evaluate accuracy, latency, memory, and unsupported-operation fallbacks.

QAT inserts fake-quantization during forward passes. Values are quantized and dequantized numerically, while trainable parameters remain floating point for gradient updates. The exported model is then converted to real low-precision operators.

6. Mathematical Foundation

Affine quantization maps a real value x to integer q :

```
q = clamp(round(x / s) + z, q_min, q_max)
x_hat = s(q - z)
```

- $s > 0$ is the scale.
- z is the integer zero-point.
- $[q_{\min}, q_{\max}]$ is the integer range, such as $[-128, 127]$ for signed INT8.
- x_{hat} is the dequantized approximation.

For asymmetric min-max quantization:

```
s = (x_max - x_min) / (q_max - q_min)
z = round(q_min - x_min / s)
```

For symmetric signed quantization:

```
a = max(|x_min|, |x_max|)
s = a / q_max
z = 0
```

Quantization error is often summarized by:

```
MSE = (1/n) * sum_i (x_i - x_hat_i)^2
SQNR = 10 log10(sum_i x_i^2 / sum_i (x_i - x_hat_i)^2)
```

For a linear layer, integer accumulation can approximate $Y = XW$:

```
Y_real approximately s_x s_w * sum_k ((q_x - z_x)(q_w - z_w))
```

Accumulation usually uses INT32 to avoid overflow. Bias is commonly represented using scale $s_x s_w$.

7. Practical Implementation

Dynamic INT8 quantization is a small, practical CPU example:

```
import time
import torch
from torch import nn

torch.manual_seed(0)

class Classifier(nn.Module):
```

```

def __init__(self):
    super().__init__()
    self.net = nn.Sequential(
        nn.Linear(128, 256),
        nn.ReLU(),
        nn.Linear(256, 10),
    )

def forward(self, x):
    return self.net(x)

fp32_model = Classifier().eval()
int8_model = torch.ao.quantization.quantize_dynamic(
    fp32_model,
    {nn.Linear},
    dtype=torch.qint8,
)

x = torch.randn(64, 128)
with torch.inference_mode():
    y_fp32 = fp32_model(x)
    y_int8 = int8_model(x)

print("mean absolute output error:", (y_fp32 - y_int8).abs().mean().item())

def benchmark(model, runs=500):
    for _ in range(20):
        model(x)
    start = time.perf_counter()
    for _ in range(runs):
        model(x)
    return 1000 * (time.perf_counter() - start) / runs

print("FP32 ms:", benchmark(fp32_model))
print("INT8 ms:", benchmark(int8_model))

```

8. Code Explanation

`quantize_dynamic` replaces supported `Linear` modules with dynamic quantized versions. Weights are stored in INT8; activations are quantized for each call. `eval()` and `inference_mode()` disable training behavior and gradient tracking. Output error checks numerical drift; the warm-up in `benchmark` avoids measuring initialization overhead.

9. Training / Evaluation

Use a representative calibration set, not random noise. It need not contain labels, but it must cover real input distributions, sequence lengths, lighting conditions, or languages. Evaluate the original task metric and deployment metrics together: accuracy/F1/perplexity, p50/p95/p99 latency, throughput, peak memory, artifact size, and energy if relevant. If PTQ loses too much quality, try percentile or entropy calibration, per-channel weights, mixed precision, SmoothQuant, or QAT.

10. Complexity and Cost

- Storage changes approximately from $4N$ bytes in FP32 to N bytes in INT8, plus small scale metadata.
- Arithmetic count is usually unchanged, but low-precision hardware processes more values per instruction.
- Calibration is much cheaper than training; QAT requires extra fine-tuning.
- Quantize/dequantize boundaries and fallback operators add overhead.
- Memory bandwidth often improves by about the bit-width ratio, but end-to-end speedup is hardware- and graph-dependent.

11. Common Use Cases

- INT8 CNNs on phones and embedded accelerators
- Quantized Transformers on CPU servers
- FP16/BF16 GPU inference and training
- 8-bit/4-bit LLM weight-only inference
- QLoRA fine-tuning with a quantized frozen base model

12. Common Mistakes

- Calibrating on unrepresentative data
- Reporting model size reduction as latency improvement without benchmarking
- Quantizing softmax, normalization, or outlier-heavy layers blindly
- Comparing models with different preprocessing or evaluation sets
- Ignoring operator fallbacks to FP32
- Using tiny benchmark runs without warm-up or synchronization
- Treating FP16 and INT8 as equally supported on every device

13. Edge Cases / Limitations

Outliers can force a large scale and waste most integer levels. Small models may be dominated by conversion overhead. Regression and generative tasks can be more sensitive than classification. Some devices lack efficient low-bit kernels. Very low precision may amplify errors across autoregressive decoding steps.

14. Variations

Variation	What changes	When to use	Importance
FP16/BF16	Floating-point exponent/mantissa shrink	GPUs; BF16 when range matters	Essential placement topic
Weight-only INT8/INT4	Activations remain higher precision	LLMs limited by weight bandwidth	Strong project/research value
QAT	Train through fake quantization	PTQ quality is insufficient	Important for deployment roles
SmoothQuant	Moves activation difficulty into weights by scaling	Transformer activation outliers	Advanced interview topic

Variation	What changes	When to use	Importance
GPTQ/AWQ	Layerwise low-bit weight quantization	4-bit LLM inference	Important for LLM roles
QLoRA	4-bit base plus trainable low-rank adapters	Memory-efficient LLM fine-tuning	High practical value

15. Related Topics

Quantization complements pruning and distillation. ONNX can carry quantized graphs; TensorRT selects precision-specific kernels. Mixed-precision training concerns numerical stability, while inference quantization emphasizes latency and memory. LoRA reduces trainable parameters; quantization reduces storage/compute representation.

16. Interview Questions

1. **What is quantization?** Mapping high-precision tensors to fewer-bit representations while controlling error.
2. **PTQ vs QAT?** PTQ converts after training; QAT exposes the model to simulated quantization during training and usually preserves accuracy better.
3. **Why is a zero-point needed?** It lets integer arithmetic represent real zero exactly in affine quantization.
4. **Symmetric vs asymmetric?** Symmetric uses a centered range and often zero zero-point; asymmetric fits skewed ranges better but adds arithmetic complexity.
5. **Per-tensor vs per-channel?** Per-channel gives each channel its own scale and usually reduces weight error at small metadata cost.
6. **Why use INT32 accumulation?** A sum of many INT8 products can exceed INT8/INT16 range.
7. **Why can INT8 be slower than FP16?** Unsupported kernels, conversion boundaries, small workloads, or hardware optimized for FP16.
8. **What makes calibration good?** Representative inputs that cover the deployment distribution and realistic shapes.
9. **What is fake quantization?** Simulated round/clamp/dequantize in the forward pass with floating-point parameters for training.
10. **How do you debug accuracy loss?** Compare layer outputs, inspect ranges/outliers, keep sensitive layers high precision, change granularity/calibration, then try QAT.
11. **Why is BF16 often safer than FP16?** BF16 has FP32-like exponent range, reducing overflow/underflow, though less mantissa precision.
12. **Does 4-bit imply an 8x speedup over FP32?** No; theoretical storage ratio does not account for kernels, unpacking, memory, or nonquantized operations.

17. Practice Tasks

- Quantize an MLP dynamically and compare size, latency, and output error.
- PTQ a ResNet on a representative CIFAR-10 calibration split.
- Compare per-tensor and per-channel weight error.
- Diagnose a model calibrated on zeros and explain the accuracy collapse.
- Extend an INT8 pipeline with mixed precision for sensitive layers.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Mobile INT8 vision app	Classifies images offline	PyTorch, ONNX/TFLite	CIFAR-10 or Food-101	End-to-end edge optimization
Quantization benchmark lab	Compares FP32/FP16/INT8/INT4	PyTorch, ONNX Runtime	GLUE subset or ImageNet subset	Measurement and systems depth
QLoRA domain assistant	Fine-tunes a 4-bit LLM	Transformers, PEFT, bitsandbytes	Domain instruction data	Modern LLM efficiency

19. Quick Revision

- **Key idea:** trade numerical precision for lower memory and faster supported kernels.
- **Main formula:** $q = \text{clamp}(\text{round}(x/s) + z)$.
- **When to use:** deployment is memory-, bandwidth-, energy-, or latency-constrained.
- **Metrics:** task quality, p95 latency, throughput, peak memory, model size.
- **Common traps:** bad calibration, unsupported ops, unrepresentative benchmarks.
- **Interview one-liner:** Quantization is useful only when accuracy stays acceptable and the target hardware has efficient low-precision execution.

20. Final Cheat Sheet

Item	Summary
Definition	Represent weights/activations with fewer bits
Input/output	FP model + calibration/training data -> low-precision model
Main steps	Observe ranges, choose mapping, convert kernels, validate
Hyperparameters	Bit width, granularity, symmetric/asymmetric, clipping method
Metrics	Accuracy/perplexity, latency percentiles, throughput, memory
Pros	Smaller, lower bandwidth, often faster/cheaper
Cons	Accuracy loss, hardware/operator dependence, calibration effort
Best use cases	Edge devices, CPU serving, large-model inference

Pruning

1. Overview

Pruning removes parameters, channels, heads, filters, blocks, or other structures judged unnecessary. The goal is a smaller or cheaper model with limited quality loss. Pruning can reduce storage, arithmetic, and memory

traffic, but only structured pruning reliably produces speedups on common dense hardware without specialized sparse kernels.

2. Intuition

A large network often has redundant pathways. Pruning is like trimming weak branches from a tree: remove low-value parts, then let the remaining network recover through fine-tuning. Removing individual leaves produces an irregular shape (unstructured sparsity); removing whole branches (structured pruning) is easier for hardware to exploit.

3. Prerequisites

Neural-network weights and gradients, regularization, fine-tuning, tensor shapes, convolutions/attention, sparsity formats, and deployment benchmarking.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Unstructured pruning	Remove individual weights	High sparsity and compression	Set small weights to zero	Why zeros may not speed dense kernels
Structured pruning	Remove channels, filters, heads, or blocks	Produces smaller dense tensors	Delete 20% of conv filters	Accuracy vs deployability
Magnitude pruning	Rank by absolute weight	Simple strong baseline	Remove smallest `	w
Gradient/sensitivity pruning	Estimate effect on loss	Better importance signal	Taylor score `	w dL/dw
Global pruning	Rank parameters across layers	Allocates sparsity adaptively	One threshold for model	Global vs layerwise
Iterative pruning	Prune and fine-tune repeatedly	Better recovery	10% per round	One-shot vs gradual
Lottery Ticket Hypothesis	Dense models contain trainable sparse subnetworks	Explains redundancy research	Reset surviving weights	Research significance
N:M sparsity	Exactly N nonzeros per M values	Hardware-friendly pattern	NVIDIA 2:4 sparsity	Structured semi-sparsity

5. Algorithm / Working Process

1. Train or obtain a baseline model.
2. Define the pruning unit: scalar weight, channel, attention head, or block.

3. Compute an importance score.
4. Choose global/layerwise sparsity and remove the least important units.
5. Fine-tune with masks enforced or physically rebuild a smaller architecture.
6. Repeat for gradual pruning if needed.
7. Export to a runtime that exploits the resulting sparsity and measure real latency.

During masked training, a binary mask m gives effective weights $w_{\text{tilde}} = m \text{ elementwise_mul } w$. A physical structured-pruning pass must also update downstream tensor dimensions.

6. Mathematical Foundation

An ideal pruning problem is:

$$\min_w L(w; D) \text{ subject to } ||w||_0 \leq k$$

$||w||_0$ counts nonzero weights. This combinatorial problem is hard, so practical methods use heuristics.

Magnitude score:

$$s_i = |w_i|$$

First-order Taylor importance approximates loss change if weight w_i is removed:

$$\Delta L_i \text{ approximately } |(dL/dw_i)(0 - w_i)| = |w_i dL/dw_i|$$

Group-lasso encourages entire groups to vanish:

$$L_{\text{total}} = L_{\text{task}} + \lambda \sum_g ||w_g||_2$$

Sparsity and density are:

$$\begin{aligned} \text{sparsity} &= \text{number_of_zeros} / \text{number_of_parameters} \\ \text{density} &= 1 - \text{sparsity} \end{aligned}$$

Theoretical dense MAC reduction from removing a fraction p of output channels in a layer is about p for that layer, plus reductions in the next layer's input dimension. Actual latency is not linear in FLOPs.

7. Practical Implementation

```
import torch
from torch import nn
```

```

from torch.nn.utils import prune

class SmallMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(100, 64)
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        return self.fc2(torch.relu(self.fc1(x)))

model = SmallMLP()

# Rank all Linear weights together and prune the smallest 40%.
parameters = [(model.fc1, "weight"), (model.fc2, "weight")]
prune.global_unstructured(
    parameters,
    pruning_method=prune.L1Unstructured,
    amount=0.40,
)

zeros = sum((module.weight == 0).sum().item() for module, _ in parameters)
total = sum(module.weight.numel() for module, _ in parameters)
print(f"masked sparsity: {zeros / total:.1%}")

# Fine-tune here while masks remain active, then make zeros permanent.
for module, name in parameters:
    prune.remove(module, name)

with torch.inference_mode():
    print(model(torch.randn(8, 100)).shape)

```

8. Code Explanation

Global L1 pruning compares magnitudes across both layers. PyTorch reparameterizes each weight as an original parameter multiplied by a mask. Fine-tuning preserves the zeros because the masked weights do not contribute. `prune.remove` removes the reparameterization and stores the masked tensor permanently; it does not automatically compress storage or select a sparse kernel.

9. Training / Evaluation

Start from a reproducible dense baseline. Use validation quality to select sparsity and a held-out test set only for final reporting. Gradual schedules commonly outperform one-shot high sparsity. Monitor layerwise sparsity, gradient flow, task metrics, parameter count, serialized size, latency, memory, and kernel utilization. Fine-tuning learning rate is typically lower than initial training.

10. Complexity and Cost

- Sorting N scores costs $O(N \log N)$; threshold selection can be $O(N)$.
- Masks add memory during pruning unless stored compactly.

- Unstructured sparse storage needs indices, so low sparsity may consume more memory than dense storage.
- Structured pruning reduces dense tensor dimensions and works on ordinary kernels.
- Iterative pruning increases training cost by multiple fine-tuning rounds.

11. Common Use Cases

Model compression for mobile vision, reducing Transformer heads/MLP width, N:M acceleration on supported GPUs, removing redundant channels before quantization, and studying sparse subnetworks.

12. Common Mistakes

- Assuming many zeros guarantee faster inference
- Pruning every layer at the same rate despite different sensitivity
- Measuring parameter sparsity but not end-to-end latency
- Forgetting to preserve masks during fine-tuning
- Physically removing a channel without updating dependent layers
- Pruning too aggressively in one step
- Comparing against a poorly trained dense baseline

13. Edge Cases / Limitations

Small/bottleneck layers can be highly sensitive. Residual additions require compatible shapes. Attention heads may interact, making individual head scores misleading. Sparse kernel overhead can dominate at small batch sizes or moderate sparsity. Dynamic input-dependent sparsity complicates batching and compilation.

14. Variations

Variation	Change and use	Importance
One-shot pruning	Prune once, then fine-tune; cheapest baseline	Placements/projects
Iterative magnitude pruning	Repeated prune-recover cycles	Research and robust compression
Channel/filter pruning	Removes dense dimensions	Most deployment-relevant
Head/layer pruning	Removes Transformer components	LLM/NLP interviews
N:M sparsity	Fixed local pattern for accelerator support	Systems roles
Movement pruning	Uses weight movement during fine-tuning	Transfer-learning research

15. Related Topics

Pruning creates sparsity; sparse models and kernels determine whether it becomes speed. Distillation trains a smaller student without requiring the same sparse topology. Quantization changes value precision and can be combined with pruning. Neural architecture search may discover a compact architecture directly.

16. Interview Questions

1. **What is pruning?** Removing low-value model components while retaining acceptable quality.
2. **Structured vs unstructured?** Structured removes whole tensor groups; unstructured removes scalar weights.
3. **Why may 90% sparsity not improve latency?** Dense kernels still process zeros unless sparse formats/kernels are used.
4. **What is magnitude pruning?** Remove weights with the smallest absolute values.
5. **Global vs layerwise pruning?** Global uses one ranking across layers; layerwise enforces a chosen rate per layer.
6. **Why fine-tune after pruning?** Remaining weights adapt to compensate for removed capacity.
7. **What is the lottery-ticket idea?** A dense random network may contain sparse subnetworks trainable to comparable accuracy.
8. **What is 2:4 sparsity?** Two nonzero values are retained in every consecutive group of four, matching some hardware.
9. **How would you prune a residual network?** Respect shape dependencies, prune linked channels consistently, and often avoid shortcut/output bottlenecks.
10. **How do you choose sparsity?** Sweep rates using validation quality and target-device latency, not a universal threshold.
11. **What is a Taylor pruning score?** A first-order estimate of loss change, often $|w * \text{gradient}|$.
12. **Can pruning reduce memory immediately?** Only if represented sparsely or rebuilt into smaller dense tensors; zeros in a dense tensor still occupy space.

17. Practice Tasks

- Globally prune an MLP and plot quality vs sparsity.
- Channel-prune a CIFAR-10 CNN and physically rebuild it.
- Compare one-shot 60% pruning with three gradual 20% rounds.
- Debug why a 70% sparse model has unchanged latency.
- Combine structured pruning with INT8 quantization.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Pruning dashboard	Sweeps sparsity and tracks quality/latency	PyTorch, Streamlit	CIFAR-10	Clear experiment design
Tiny detector	Channel-prunes an object detector	PyTorch, ONNX	Pascal VOC	Deployment-focused CV
Sparse Transformer study	Compares head, block, and magnitude pruning	Transformers, PyTorch	AG News/GLUE	NLP research signal

19. Quick Revision

- **Key idea:** remove redundant parameters or structures.
- **Main formula:** $\min L(w)$ subject to $\|w\|_0 \leq k$.

- **When to use:** a trained model has excess capacity and deployment cost matters.
- **Metrics:** quality, sparsity, parameter count, latency, memory.
- **Common traps:** equating zeros/FLOPs with speed.
- **Interview one-liner:** Structured pruning is less flexible but more likely to yield real hardware speedups.

20. Final Cheat Sheet

Item	Summary
Definition	Remove low-importance model components
Input/output	Dense model -> masked sparse or smaller dense model
Main steps	Score, prune, fine-tune, export, benchmark
Hyperparameters	Sparsity, granularity, schedule, score, fine-tuning LR
Metrics	Accuracy, sparsity, size, p95 latency, throughput
Pros	Smaller model; possible speed/energy gains
Cons	Recovery training; sparse hardware dependence
Best use cases	Overparameterized models and supported sparsity patterns

Knowledge Distillation

1. Overview

Knowledge distillation (KD) trains a smaller student model to imitate a larger or ensemble teacher. The student learns from teacher probabilities, features, attention maps, or relations in addition to hard labels. Distillation can retain more quality than training the small model alone and is used in compressed vision models, speech systems, BERT variants, and LLM instruction/self-distillation.

2. Intuition

A hard label says an image is a cat. A teacher distribution might say **cat 0.72**, **fox 0.20**, **dog 0.07**, revealing which classes look similar. That “dark knowledge” teaches the student a smoother decision boundary than the one-hot label alone.

3. Prerequisites

Classification logits, softmax, cross-entropy, KL divergence, temperature, teacher/student architectures, feature maps, and supervised training loops.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Response distillation	Match output distributions	Architecture-agnostic baseline	Student matches teacher logits	Temperature and KL

Subtopic	Meaning	Why it matters	Example	Interview angle
Feature distillation	Match intermediate representations	Transfers internal structure	Match CNN feature maps	Dimension alignment
Relation distillation	Match pairwise similarities/distances	Preserves geometry	Pairwise embedding distances	Beyond pointwise targets
Online distillation	Teacher and student train together	No fixed pretrained teacher	Peer networks teach each other	Stability risks
Self-distillation	Same architecture/generation teaches itself	Can improve regularization	Earlier checkpoint or deeper layer teaches	Why it can help without compression
Data-free distillation	Uses synthetic inputs when data is unavailable	Privacy/legacy settings	Generate calibration images	Distribution fidelity
Sequence-level KD	Train on teacher-generated sequences	Useful for translation/LLMs	Teacher summaries as targets	Exposure to teacher errors

5. Algorithm / Working Process

1. Train or select a strong teacher and freeze it.
2. Choose a smaller student that meets deployment constraints.
3. For each batch, compute teacher logits without gradients.
4. Divide teacher and student logits by temperature T .
5. Minimize a weighted combination of hard-label loss and soft-target loss.
6. Optionally align features using projections and auxiliary losses.
7. Validate the student independently and benchmark its actual deployment cost.

Input is the same sample for teacher and student. Output is the student's prediction at inference; the teacher is discarded.

6. Mathematical Foundation

Temperature-softened class probabilities are:

$$p_i^{T} = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

For $T > 1$, the distribution becomes softer and reveals relative similarities. A common objective is:

```
L = alpha * CE(y, p_student)
  + (1 - alpha) * T^2 * KL(p_teacher^T || p_student^T)
```

The T^2 factor compensates for gradients shrinking approximately as $1/T^2$. KL divergence is:

```
KL(p || q) = sum_i p_i log(p_i / q_i)
```

Feature matching can use:

```
L_feature = (1/N) ||g(h_student) - h_teacher||_2^2
```

where projection g aligns dimensions. Total loss may be $L + \text{beta } L_{\text{feature}}$.

7. Practical Implementation

```
import torch
import torch.nn.functional as F
from torch import nn

def distillation_loss(student_logits, teacher_logits, labels,
                      temperature=4.0, alpha=0.3):
    hard_loss = F.cross_entropy(student_logits, labels)
    soft_targets = F.softmax(teacher_logits / temperature, dim=-1)
    soft_log_probs = F.log_softmax(student_logits / temperature, dim=-1)
    soft_loss = F.kl_div(
        soft_log_probs,
        soft_targets,
        reduction="batchmean",
    ) * temperature**2
    return alpha * hard_loss + (1 - alpha) * soft_loss

teacher = nn.Sequential(nn.Linear(32, 128), nn.ReLU(), nn.Linear(128, 5)).eval()
student = nn.Sequential(nn.Linear(32, 24), nn.ReLU(), nn.Linear(24, 5))
optimizer = torch.optim.AdamW(student.parameters(), lr=1e-3)

x = torch.randn(64, 32)
labels = torch.randint(0, 5, (64,))

with torch.no_grad():
    teacher_logits = teacher(x)

student_logits = student(x)
loss = distillation_loss(student_logits, teacher_logits, labels)
optimizer.zero_grad()
loss.backward()
```

```
optimizer.step()  
print("KD loss:", loss.item())
```

8. Code Explanation

The teacher is in evaluation mode and runs under `no_grad`, preventing teacher updates and saving activation memory. `cross_entropy` learns ground truth. `kl_div` compares softened teacher probabilities with the student's log probabilities. Only student parameters are optimized. In a real pipeline, the teacher must first be trained or loaded from a strong checkpoint.

9. Training / Evaluation

Use identical preprocessing and label semantics for both models. Split normally into train/validation/test; do not tune temperature or `alpha` on the test set. Compare student-from-scratch, KD student, and teacher. Track task quality, calibration, teacher-student agreement, model size, latency, and memory. Tune `T`, `alpha`, learning rate, and feature-loss weights. A weak or biased teacher caps the usefulness of KD.

10. Complexity and Cost

Training costs more because both teacher and student perform forward passes; feature distillation also stores intermediate tensors. Teacher backward is unnecessary for offline KD. Inference cost is only the student, so deployment may be dramatically cheaper. Sequence-level distillation can require expensive teacher generation once but cache the outputs.

11. Common Use Cases

DistilBERT-style NLP compression, compact image classifiers/detectors, speech recognition on devices, ensemble-to-single-model compression, LLM instruction data generation, and transferring multimodal model knowledge.

12. Common Mistakes

- Training from an unvalidated teacher
- Forgetting `no_grad()` or teacher `eval()`
- Reversing KL arguments
- Omitting the T^2 scale and then mis-tuning weights
- Using incompatible teacher/student tokenizers or class orders
- Matching raw features with different shapes without projection
- Reporting only accuracy and not student deployment gains
- Letting synthetic teacher outputs leak test information

13. Edge Cases / Limitations

The student may lack capacity to imitate the teacher. A teacher can transfer bias, hallucinations, and label noise. High temperature can wash out useful signal; low temperature approaches hard predictions. Feature alignment may overconstrain architecturally different students. In domain shift, teacher confidence can be misleading.

14. Variations

Variation	What changes / when to use	Importance
Logit KD	Match class distributions; default baseline	Essential
Feature KD	Match hidden features when internal knowledge matters	Projects/research
Attention transfer	Match spatial/attention patterns	CV/Transformer research
Self-distillation	Earlier/deeper model teaches same family	Useful regularizer
Ensemble distillation	Average several teachers	Strong quality, expensive target creation
Sequence-level KD	Use teacher-generated text	Essential for generation roles
Progressive KD	Distill through intermediate-size students	Large teacher-student capacity gap

15. Related Topics

Label smoothing uses generic soft targets; KD uses data-dependent teacher targets. Pseudo-labeling emphasizes unlabeled data, while KD emphasizes imitation. Pruning/quantization alter an existing model; KD trains a new compact architecture. Fine-tuning transfers from pretrained weights, whereas KD transfers behavior.

16. Interview Questions

1. **What is knowledge distillation?** Training a student using signals from a stronger teacher.
2. **Why are soft labels useful?** They expose relative class similarities absent from one-hot labels.
3. **What does temperature do?** Higher temperature softens the probability distribution.
4. **Why multiply KL loss by T^2 ?** To restore gradient scale reduced by temperature.
5. **Why keep hard-label loss?** The teacher can be wrong; ground truth anchors learning.
6. **Can student outperform teacher?** Sometimes, due to regularization, data augmentation, or evaluation variance, though not guaranteed.
7. **What if teacher and student feature sizes differ?** Use a learned projection or distill invariant relations/logits.
8. **Offline vs online KD?** Offline uses a fixed pretrained teacher; online trains peers/teacher jointly.
9. **How do you choose α and T ?** Tune on validation data; larger T reveals more class structure but may weaken targets.
10. **What is self-distillation?** A model or its later/deeper component teaches another version of the same family.
11. **Why cache teacher outputs?** To avoid repeated expensive teacher inference when augmentation does not change inputs incompatibly.
12. **Main production risk?** Student faithfully inherits teacher biases/errors while appearing cheaper and easier to deploy broadly.

17. Practice Tasks

- Distill a wide MLP into a narrow MLP and compare against scratch training.
- Distill ResNet-50 into ResNet-18 on CIFAR-100.
- Sweep T and plot entropy, accuracy, and calibration.
- Debug a KD loop where the teacher remains in dropout-enabled train mode.
- Add intermediate feature matching with a projection layer.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Distilled text classifier	Compresses BERT to a small student	Transformers, PyTorch	AG News/SST-2	NLP deployment evidence
Edge vision student	Distills a large CNN into MobileNet	torchvision, ONNX	CIFAR-100/Food-101	Compression + edge story
KD experiment tracker	Compares logit/feature/relation KD	PyTorch, MLflow	Tiny ImageNet	Research methodology

19. Quick Revision

- **Key idea:** student learns teacher behavior plus labels.
- **Main formula:** $\alpha \text{ CE} + (1-\alpha) T^2 \text{ KL}$.
- **When to use:** a strong expensive model exists but deployment needs a smaller one.
- **Metrics:** student task quality, agreement, calibration, latency, memory.
- **Common traps:** weak teacher, KL direction, incompatible preprocessing.
- **Interview one-liner:** KD converts teacher confidence structure into a training signal and discards the teacher at inference.

20. Final Cheat Sheet

Item	Summary
Definition	Transfer behavior from teacher to student
Input/output	Training samples + teacher outputs -> trained student
Main steps	Teacher forward, soften logits, combine KD and label loss, update student
Hyperparameters	Temperature, hard/soft loss weight, feature-loss weight
Metrics	Accuracy/F1/perplexity, calibration, latency, size
Pros	Better compact-model quality; flexible architectures
Cons	Costly training; teacher errors transfer
Best use cases	Strong teacher available, cheap inference required

ONNX

1. Overview

ONNX (Open Neural Network Exchange) is an open graph format for representing ML models independently of the training framework. Exporters translate PyTorch or other framework models to an ONNX graph; runtimes such as ONNX Runtime, TensorRT, OpenVINO, and mobile engines optimize and execute it. ONNX is an interchange contract, not itself a universal optimizer or hardware accelerator.

2. Intuition

ONNX is like a common shipping container. PyTorch packs the model into a standardized graph; different inference engines can unpack and run it. The container describes operations, tensors, attributes, constants, and input/output contracts, but the receiving runtime must support those operations.

3. Prerequisites

Computation graphs, tensor shapes/dtypes, model train/eval modes, tracing vs control flow, serialization, inference runtimes, and numerical validation.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Graph	Nodes connected by tensors	Portable computation	Conv -> ReLU -> Pool	Static graph benefits
Operator set (opset)	Versioned operator specification	Exporter/runtime compatibility	Opset 17	Why newest is not always best
Initializer	Stored parameter/constant	Carries weights	Convolution kernel	Parameters vs graph inputs
Shape inference	Derive intermediate shapes	Enables optimization/validation	Infer matmul output	Dynamic shape limits
Dynamic axes/shapes	Dimensions vary at runtime	Supports variable batch/sequence	Batch N , sequence S	Optimization trade-off
Custom operator	Nonstandard operation implementation	Handles unsupported logic	Custom NMS op	Portability cost
Execution provider	Backend running graph segments	Maps to CPU/GPU/accelerator	CUDA, TensorRT EP	Fallback behavior
Graph optimization	Constant folding, fusion, elimination	Lowers runtime cost	Fuse Conv+BN	Runtime vs exporter optimization

5. Algorithm / Working Process

1. Put the source model in evaluation mode and prepare representative example inputs.
2. Export the computation to ONNX with suitable opset and dynamic dimensions.
3. Run ONNX model validation and shape inference.

4. Create a runtime session with desired execution providers.
5. Compare output shapes and numeric values against the source framework.
6. Benchmark on realistic shapes, including warm-up and synchronization.
7. Package preprocessing, labels/tokenizer, and input/output schema alongside the graph.

At inference, the runtime loads the graph, partitions it across providers, applies optimizations, allocates buffers, executes nodes, and returns named outputs.

6. Mathematical Foundation

An ONNX graph is a directed computation:

```
h_0 = x
h_{k+1} = op_k(h_k, parameters_k, attributes_k)
y = h_K
```

Correct export requires functional equivalence within numerical tolerance:

```
max_abs_error = max_i |y_framework_i - y_onnx_i|
relative_error_i = |a_i - b_i| / max(|b_i|, epsilon)
```

Validation commonly uses:

```
|a_i - b_i| <= atol + rtol * |b_i|
```

Shape constraints for matrix multiplication illustrate why symbolic dimensions matter:

```
A: [..., M, K], B: [..., K, N] -> C: [..., M, N]
```

ONNX defines operator semantics; it does not define a loss function or training objective for ordinary inference export.

7. Practical Implementation

```
import numpy as np
import torch
from torch import nn
import onnx
import onnxruntime as ort

class Encoder(nn.Module):
    def __init__(self):
        super().__init__()
```

```

        self.fc = nn.Linear(16, 4)

    def forward(self, x):
        return torch.softmax(self.fc(x), dim=-1)

model = Encoder().eval()
example = torch.randn(2, 16)
path = "encoder.onnx"

torch.onnx.export(
    model,
    (example,),
    path,
    input_names=["features"],
    output_names=["probabilities"],
    dynamic_axes={
        "features": {0: "batch"},
        "probabilities": {0: "batch"},
    },
    opset_version=17,
)

onnx_model = onnx.load(path)
onnx.checker.check_model(onnx_model)

session = ort.InferenceSession(path, providers=["CPUExecutionProvider"])
x = np.random.randn(3, 16).astype(np.float32)
onnx_output = session.run(["probabilities"], {"features": x})[0]

with torch.inference_mode():
    torch_output = model(torch.from_numpy(x)).numpy()

np.testing.assert_allclose(onnx_output, torch_output, rtol=1e-4, atol=1e-5)
print(onnx_output.shape)

```

8. Code Explanation

The example input establishes dtype, rank, and concrete non-dynamic dimensions. `dynamic_axes` allows batch size to vary. The checker validates graph structure and operator schemas. ONNX Runtime takes NumPy arrays keyed by exported input names. `assert_allclose` verifies semantic parity rather than assuming successful export means correct behavior.

9. Training / Evaluation

ONNX is primarily used for inference deployment, so evaluate exported artifacts against a trusted framework model on a golden dataset. Cover minimum, typical, and maximum supported shapes; empty sequences if permitted; and all output branches. Track task metrics as well as numerical error, latency percentiles, throughput, peak memory, provider assignment, and startup time.

10. Complexity and Cost

Graph execution complexity mirrors the represented model, but fusion and backend kernels can reduce constant factors. Export is usually a one-time cost. Dynamic shapes may limit memory planning and kernel specialization. Model size is mostly weight storage; external data files may be needed for models beyond protobuf size limits. Session initialization can be significant, so reuse sessions.

11. Common Use Cases

PyTorch-to-C++ deployment, cross-platform CPU/GPU inference, model serving in ONNX Runtime, conversion to TensorRT/OpenVINO, browser or mobile inference, and standardized model validation pipelines.

12. Common Mistakes

- Exporting in training mode with dropout or mutable batch normalization
- Using an opset unsupported by the target runtime
- Marking all axes dynamic unnecessarily
- Omitting preprocessing/tokenizer contracts
- Checking only that export succeeds, not numerical parity
- Accidentally executing unsupported nodes on CPU fallback
- Benchmarking session creation on every request
- Depending on Python control flow that tracing cannot capture correctly

13. Edge Cases / Limitations

Framework-specific operations may lack ONNX equivalents. Data-dependent control flow and dynamic containers can be difficult to export. Runtime support varies by opset and provider. Custom operators reduce portability. Numerically equivalent graph rewrites may create small differences, especially in FP16. Very dynamic models can lose optimization opportunities.

14. Variations

Variation	What changes / when to use	Importance
Static-shape ONNX	Fixed dimensions maximize optimization	Deployment essential
Dynamic-shape ONNX	Symbolic batch/sequence dimensions	Needed for variable inputs
Quantized ONNX	Q/DQ nodes or integer operators	CPU/edge optimization
ONNX Runtime graph format	ORT-optimized artifact	Runtime-specific deployment
Custom ops	Add missing backend functionality	Advanced systems topic
Training graphs	ONNX representations used in training tooling	Less common in placements

15. Related Topics

ONNX is a format; ONNX Runtime is an execution engine. TensorRT can consume ONNX and build an optimized NVIDIA engine. TorchScript/[torch.export](#) stay closer to PyTorch ecosystems. TFLite targets TensorFlow/mobile workflows. Quantization can appear as quantize/dequantize nodes in ONNX graphs.

16. Interview Questions

1. **What is ONNX?** A framework-neutral, versioned graph representation for ML models.
2. **Is ONNX an inference engine?** No; a runtime such as ONNX Runtime executes it.
3. **What is an opset?** A versioned collection of operator definitions and semantics.
4. **Why provide example inputs?** Export needs concrete ranks, dtypes, and paths through model logic.
5. **What are dynamic axes?** Symbolic dimensions allowed to change between inference calls.
6. **Why can export succeed but results be wrong?** Traced control flow, preprocessing mismatch, unsupported semantics, or numerical rewrites.
7. **What is an execution provider?** A backend that executes supported graph nodes on particular hardware.
8. **How do you detect CPU fallback?** Inspect provider/node assignment and runtime profiling logs.
9. **How do you validate ONNX?** Structural checker plus output and task-metric comparison across representative edge cases.
10. **Why not make every dimension dynamic?** It restricts specialization, fusion, and memory planning.
11. **How are unsupported operators handled?** Rewrite them, decompose them, use a custom op, or choose another runtime/opset.
12. **ONNX vs TensorRT?** ONNX stores a portable graph; TensorRT builds optimized NVIDIA-specific engines.

17. Practice Tasks

- Export an MLP with dynamic batch and validate it in ONNX Runtime.
- Export a CNN and inspect graph nodes with Netron.
- Benchmark static vs dynamic shapes.
- Diagnose an export mismatch caused by dropout/train mode.
- Add a parity suite for three batch sizes and two sequence lengths.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Portable inference API	Serves one ONNX model across CPU/GPU	ONNX Runtime, FastAPI	Any trained classifier	Deployment fundamentals
Export validator	Automates parity and shape tests	PyTorch, ONNX, pytest	Model zoo samples	Reliability engineering
Provider benchmark	Compares CPU/CUDA/TensorRT EPs	ONNX Runtime	ImageNet subset	Hardware-aware profiling

19. Quick Revision

- **Key idea:** portable, versioned computation graph.
- **Main equation:** runtime output should satisfy `allclose(framework, ONNX)`.
- **When to use:** separate training framework from deployment runtime.
- **Metrics:** correctness tolerance, task metric, latency, throughput, fallback count.
- **Common traps:** wrong opset, dynamic overuse, hidden preprocessing mismatch.

- **Interview one-liner:** ONNX standardizes the graph; the execution provider determines actual acceleration.

20. Final Cheat Sheet

Item	Summary
Definition	Portable ML graph and operator specification
Input/output	Framework model + examples -> <code>.onnx</code> ; runtime inputs -> outputs
Main steps	Export, check, validate numerics, select provider, benchmark
Hyperparameters	Opset, dynamic dimensions, optimization level, providers
Metrics	Parity error, task quality, latency, memory
Pros	Portability, ecosystem, graph optimization
Cons	Operator gaps, version/provider variance, dynamic-shape complexity
Best use cases	Cross-framework and production inference deployment

TensorRT Basics

1. Overview

NVIDIA TensorRT is an inference SDK and optimizing compiler/runtime for NVIDIA GPUs. It imports graphs (commonly ONNX), selects kernels ("tactics"), fuses operations, chooses precision, plans memory, and produces an engine specialized for a GPU and shape/precision configuration. It is used for low-latency vision, speech, recommendation, and Transformer inference.

TensorRT is not a training framework. An engine is less portable than ONNX: it is tied to TensorRT/CUDA compatibility, GPU capabilities, plugins, and build configuration.

2. Intuition

An ONNX graph is a recipe. TensorRT acts like a chef who knows one kitchen extremely well: it combines steps, picks the fastest tool for each input size, reuses counter space, and cooks in FP16/INT8 where safe. The resulting engine is fast but specialized to that kitchen.

3. Prerequisites

CUDA/GPU basics, ONNX graphs, tensor layouts, FP32/FP16/INT8, dynamic shapes, asynchronous execution, host/device memory, and reliable benchmarking.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Common interview angle
Builder	Compiles network into an engine	Performs expensive optimization	Build from ONNX	Build time vs runtime
Engine	Serialized optimized plan	Production inference artifact	<code>.engine</code> file	Portability limits
Execution context	Mutable inference state	Supports concurrent executions	One context per stream	Thread safety
Tactic	Candidate kernel implementation	Major performance source	Choose GEMM algorithm	Timing cache
Optimization profile	Min/opt/max dynamic shapes	Bounds runtime shapes	Batch 1/8/32	Why <code>opt</code> shape matters
Precision	FP32, FP16, BF16, INT8, etc.	Performance/accuracy trade-off	FP16 convolution	Mixed precision
Layer fusion	Combine compatible operations	Less launch/memory overhead	Conv+Bias+ReLU	Graph-level optimization
Plugin	Custom TensorRT layer	Supports missing/specialized ops	Efficient NMS	Maintenance burden
CUDA stream	Ordered asynchronous GPU work	Enables overlap/concurrency	Async H2D, execute, D2H	Synchronization correctness

5. Algorithm / Working Process

1. Export and validate an ONNX model.
2. Parse it into a TensorRT network definition.
3. Configure workspace/memory pool, allowed precisions, and optimization profiles.
4. For INT8, provide calibration or explicit quantization information.
5. Build: TensorRT profiles tactics and applies graph optimizations.
6. Serialize the engine and record its exact environment/configuration.
7. At runtime, deserialize once, create execution contexts, allocate buffers, set shapes, and enqueue inference on CUDA streams.
8. Validate accuracy and benchmark after warm-up with explicit synchronization.

Inputs are device tensors matching a profile. Processing uses the compiled engine. Outputs remain device tensors unless copied to host.

6. Mathematical Foundation

TensorRT does not change the intended model function $y = f_{\text{theta}}(x)$, but chooses an implementation minimizing measured or estimated cost:

```
tactic* = argmin_t runtime(t | layer, shape, dtype, GPU)
```

For batch B , throughput is:

```
throughput = B / batch_time
per_item_latency_if_amortized = batch_time / B
```

The latter is not request latency when a request waits to form a batch.

Arithmetic intensity helps explain performance:

```
arithmetic_intensity = FLOPs / bytes_moved
achievable_performance <= min(peak_compute,
                               memory_bandwidth * arithmetic_intensity)
```

FP16/INT8 help most when supported tensor-core kernels and memory behavior match the workload.

7. Practical Implementation

The CLI is the clearest first deployment and benchmark path:

```
# Build and benchmark an FP16 engine for a fixed image shape.
trtexec \
  --onnx=model.onnx \
  --saveEngine=model_fp16.engine \
  --fp16 \
  --shapes=input:8x3x224x224 \
  --warmUp=1000 \
  --duration=10

# Dynamic shapes require a bounded optimization profile.
trtexec \
  --onnx=model_dynamic.onnx \
  --saveEngine=model_dynamic.engine \
  --fp16 \
  --minShapes=input:1x3x224x224 \
  --optShapes=input:8x3x224x224 \
  --maxShapes=input:32x3x224x224
```

Minimal Python build logic (API details can vary by TensorRT release):

```
import tensorrt as trt

LOGGER = trt.Logger(trt.Logger.WARNING)
```

```
def build_engine(onnx_path, engine_path):
    builder = trt.Builder(LOGGER)
    network = builder.create_network(
        1 << int(trt.NetworkDefinitionCreationFlag.EXPLICIT_BATCH)
    )
    parser = trt.OnnxParser(network, LOGGER)
    config = builder.create_builder_config()
    config.set_flag(trt.BuilderFlag.FP16)

    with open(onnx_path, "rb") as model_file:
        if not parser.parse(model_file.read()):
            errors = [str(parser.get_error(i)) for i in range(parser.num_errors)]
            raise RuntimeError("ONNX parse failed:\n" + "\n".join(errors))

    serialized = builder.build_serialized_network(network, config)
    if serialized is None:
        raise RuntimeError("TensorRT engine build failed")
    with open(engine_path, "wb") as engine_file:
        engine_file.write(serialized)
```

8. Code Explanation

`trtexec` exposes build and runtime metrics without requiring application code and should be used to establish an engine-only baseline. The Python parser reports every unsupported/invalid node instead of silently failing. Explicit batch makes batch a graph dimension. Production code must additionally create a runtime/context, set dynamic shapes, manage CUDA buffers, enqueue work, and synchronize only when results are needed.

9. Training / Evaluation

Training is external. Evaluate the source model, ONNX model, and TensorRT engine on identical preprocessed examples. For FP16/INT8, verify task metrics and layer/output tolerances. Benchmark each target shape, concurrency level, and precision after warm-up. Record GPU model, clocks/power mode, TensorRT/CUDA versions, transfer inclusion, batch size, p50/p95/p99, throughput, and memory.

10. Complexity and Cost

Engine building can take seconds to minutes because tactics are profiled. Runtime complexity follows model operations but benefits from fusion and optimized kernels. Larger workspaces may enable faster tactics. Multiple profiles and contexts consume memory. Host-device transfer can dominate small models; keep preprocessing/postprocessing and tensors on GPU where practical.

11. Common Use Cases

Real-time object detection, autonomous systems, video analytics, GPU microservices, speech inference, recommendation ranking, and Transformer/LLM inference through TensorRT-LLM.

12. Common Mistakes

- Treating ONNX export success as TensorRT correctness
- Rebuilding an engine for every request
- Reusing one execution context unsafely across concurrent streams
- Omitting optimization profiles for dynamic inputs
- Choosing unrealistic min/opt/max shapes
- Timing asynchronous GPU calls without synchronization
- Including transfers in one benchmark but excluding them in another
- Deploying an engine built for an incompatible environment
- Enabling FP16/INT8 without evaluating accuracy

13. Edge Cases / Limitations

Unsupported operators require graph rewrites or plugins. Dynamic/data-dependent shapes reduce specialization. Engines have compatibility constraints across GPUs and software versions. Very small models can be launch- or transfer-bound. Plugins must implement serialization, shape inference, dtype support, and efficient kernels.

14. Variations

Variation	What changes / when to use	Importance
FP16 engine	Faster tensor-core compute with modest precision loss	Essential
INT8 engine	Calibration/explicit Q/DQ; highest common compression	Essential deployment topic
Dynamic profiles	Bounded shape ranges	Production variable input
TensorRT execution provider	ONNX Runtime delegates graph segments	Easier integration
TensorRT-LLM	LLM-specific kernels, parallelism, KV cache	LLM systems roles
Custom plugins	Implement unsupported/fused operation	Advanced GPU roles

15. Related Topics

ONNX provides an import graph; TensorRT compiles it. Quantization controls precision. CUDA streams and custom kernels implement execution. Batching raises utilization. Triton Inference Server can serve TensorRT engines with dynamic batching and multiple instances; it is distinct from the Triton kernel language.

16. Interview Questions

1. **What does TensorRT do?** Compiles and executes optimized inference graphs on NVIDIA GPUs.
2. **What is a tactic?** A candidate kernel/algorithm implementation selected for a layer and shape.
3. **Why are engines not universally portable?** They depend on GPU capabilities, TensorRT/CUDA versions, plugins, and build settings.
4. **What is an optimization profile?** Min/opt/max bounds for dynamic input dimensions.
5. **Why is the opt shape important?** Tactic selection is optimized around it.

6. **Builder vs runtime?** Builder performs expensive compilation; runtime deserializes and executes the engine.
7. **Why use multiple execution contexts?** Contexts hold mutable state and enable safe concurrent inferences.
8. **How do you time GPU inference?** Warm up, use CUDA events or synchronize around timed work, and state whether transfers are included.
9. **What if ONNX has an unsupported op?** Rewrite/decompose it, use a supported equivalent, or implement a plugin.
10. **What is layer fusion?** Combining operations to reduce memory traffic and kernel launches.
11. **Does FP16 always improve accuracy-neutral latency?** No; support, shape, precision sensitivity, and surrounding overhead matter.
12. **Why use a timing cache?** Reuse tactic measurements to reduce repeated engine build time.

17. Practice Tasks

- Build FP32 and FP16 engines with `trtexec` and compare them.
- Create profiles for batches 1–32 and test invalid shapes.
- Profile transfer-inclusive vs engine-only latency.
- Diagnose a dynamic engine whose batch-1 performance is poor.
- Compare ONNX Runtime CUDA and TensorRT providers.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
TensorRT vision service	Optimized GPU classification endpoint	PyTorch, ONNX, TensorRT, FastAPI	Food-101	End-to-end GPU deployment
Dynamic-shape profiler	Explores profiles, precision, concurrency	TensorRT, CUDA/Python	Synthetic + ImageNet subset	Performance engineering
Detection plugin pipeline	Integrates efficient decode/NMS	TensorRT, C++/CUDA	COCO subset	Advanced deployment work

19. Quick Revision

- **Key idea:** compile a graph into GPU- and shape-specialized inference.
- **Main choice:** tactic minimizing runtime under dtype/shape constraints.
- **When to use:** NVIDIA GPU inference where latency/throughput matter.
- **Metrics:** p95/p99, throughput, GPU memory, accuracy, build time.
- **Common traps:** async timing, profile mismatch, portability, fallbacks.
- **Interview one-liner:** TensorRT speed comes from specialization, fusion, precision, and tactic selection —not merely ONNX conversion.

20. Final Cheat Sheet

Item	Summary
Definition	NVIDIA inference compiler and runtime
Input/output	Network/ONNX -> serialized engine -> GPU outputs
Main steps	Parse, configure, profile tactics, build, deserialize, enqueue
Hyperparameters	Precision, profiles, workspace, tactics, concurrency
Metrics	Quality, latency percentiles, throughput, GPU memory
Pros	Excellent NVIDIA performance, fusion, mixed precision
Cons	Build cost, compatibility constraints, NVIDIA-specific
Best use cases	Production GPU inference with known workloads

Batch Inference

1. Overview

Batch inference processes multiple samples in one model invocation. It amortizes dispatch overhead, improves vectorization and GPU utilization, and often increases throughput. Offline batch inference processes a finite dataset; online dynamic batching groups requests arriving within a short window. Batching can increase individual request latency, padding waste, and memory use.

2. Intuition

A bus moves many passengers more efficiently than separate cars, but a passenger may wait for the bus to fill. Similarly, a GPU performs large matrix operations efficiently, while queueing requests to form a batch adds delay.

3. Prerequisites

Tensor batch dimensions, dataloaders, padding/masking, GPU memory, queues, latency percentiles, throughput, and asynchronous serving.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Static batching	Fixed batch size	Predictable and simple	Offline batch 128	Last partial batch
Dynamic batching	Group requests at runtime	Higher online utilization	Wait up to 2 ms for batch	Delay vs throughput
Micro-batching	Split a logical large batch	Fits memory/pipeline stages	1024 items as 8x128	Different from gradient accumulation

Subtopic	Meaning	Why it matters	Example	Interview angle
Padding and masks	Equalize variable shapes	Enables tensor batching	Pad sequences to max length	Padding waste
Bucketing	Group similar lengths/shapes	Reduces padding	Length buckets 64/128/256	Scheduling trade-off
Maximum batch size	Hard resource/policy bound	Controls OOM and tail latency	<code>max_batch=32</code>	How to tune
Timeout/window	Maximum queue delay	Controls online SLA	2 ms batching window	Tail latency effect
Ragged batching	Represent variable lengths compactly	Avoids padding in supported kernels	Packed tokens	Runtime support

5. Algorithm / Working Process

Offline:

1. Read and preprocess samples in parallel.
2. Collate compatible samples, pad if needed, and create masks.
3. Transfer the batch to the accelerator.
4. Run one inference call.
5. Split outputs back into per-sample records and preserve identifiers/order.
6. Handle final partial batch and failures idempotently.

Online dynamic batching:

1. Requests enter a queue with deadlines.
2. Scheduler groups compatible shapes until `max_batch_size` or `max_wait`.
3. Execute the batch.
4. Demultiplex outputs and resolve individual responses.

6. Mathematical Foundation

For batch size `B` and batch service time `T(B)`:

```
throughput(B) = B / T(B)
compute_time_per_item(B) = T(B) / B
```

Online request latency includes:

```
L_request = L_queue + L_preprocess + T(B) + L_postprocess + L_network
```

Padding efficiency for sequence lengths `l_i` padded to `L_max` is:

```

efficiency = sum_i l_i / (B * L_max)
waste = 1 - efficiency

```

Little's Law connects average concurrency N , arrival rate λ , and response time W in a stable system:

$$N = \lambda W$$

If arrivals exceed sustainable throughput, the queue grows and latency becomes unbounded.

7. Practical Implementation

```

from time import perf_counter
import torch
from torch import nn
from torch.utils.data import DataLoader, TensorDataset

device = "cuda" if torch.cuda.is_available() else "cpu"
model = nn.Sequential(nn.Linear(512, 1024), nn.ReLU(), nn.Linear(1024, 10))
model = model.to(device).eval()

features = torch.randn(10_000, 512)
ids = torch.arange(len(features))
loader = DataLoader(
    TensorDataset(ids, features),
    batch_size=256,
    shuffle=False,
    num_workers=0,
    pin_memory=(device == "cuda"),
)

predictions = []
start = perf_counter()
with torch.inference_mode():
    for batch_ids, batch_x in loader:
        batch_x = batch_x.to(device, non_blocking=True)
        batch_pred = model(batch_x).argmax(dim=-1).cpu()
        predictions.extend(zip(batch_ids.tolist(), batch_pred.tolist()))

if device == "cuda":
    torch.cuda.synchronize()
elapsed = perf_counter() - start
print(f"processed={len(predictions)}, samples/s={len(predictions) / elapsed:.1f}")
assert [item_id for item_id, _ in predictions] == list(range(len(features)))

```

8. Code Explanation

`DataLoader` forms fixed-size batches and keeps sample IDs so outputs remain traceable. `inference_mode` removes autograd overhead. Pinned memory plus nonblocking copies can enable transfer overlap on CUDA when the surrounding pipeline supports it. Final synchronization ensures GPU work is included in timing. The assertion catches output reordering or dropped examples.

9. Training / Evaluation

No training change is required, but batch-dependent layers must be in evaluation mode. Sweep batch sizes rather than assuming the largest wins. Measure steady-state throughput, per-request p50/p95/p99, maximum memory, queue delay, padding ratio, and OOM rate. Use real input-length distributions and production concurrency.

10. Complexity and Cost

The arithmetic work is roughly B times per-sample work, but parallel hardware reduces wall time. Activation memory generally scales approximately with B and sequence/image dimensions. CPU preprocessing, transfers, and postprocessing can bottleneck the GPU. Larger batches can reduce launch overhead but increase cache pressure and tail latency.

11. Common Use Cases

Dataset embedding generation, nightly scoring, image/video processing, recommendation ranking, LLM prompt batching, ETL enrichment, and online inference servers with dynamic batching.

12. Common Mistakes

- Calling `model()` once per row inside a data loop
- Dropping or duplicating IDs when reassembling outputs
- Shuffling offline inference unintentionally
- Padding every sequence to a global maximum
- Increasing batch size until OOM without measuring the optimum
- Reporting $T(B)/B$ as online request latency
- Timing asynchronous CUDA without synchronization
- Ignoring final partial batches and retry semantics

13. Edge Cases / Limitations

Variable shapes may be incompatible. Autoregressive decoding requests progress at different speeds. A single very long sequence can inflate padding for the whole batch. Stateful models may not batch cleanly. Strict low-latency workloads cannot afford long batching windows. Large batches can starve other GPU tenants.

14. Variations

Variation	What changes / when to use	Importance
Offline static batching	No request deadlines; maximize throughput	Essential
Online dynamic batching	Short queue window	Serving interviews
Length bucketing	Similar shapes grouped	NLP/LLM essential

Variation	What changes / when to use	Importance
Continuous batching	LLM sequences join/leave between decode steps	Advanced LLM serving
Micro-batching	Bound memory or pipeline-stage work	Distributed systems
Adaptive batching	Window/size follows load and SLA	Production/research

15. Related Topics

Latency vs throughput determines batch tuning. Caching can avoid some batch work. TensorRT optimization profiles must cover batch shapes. KV caching plus continuous batching dominates LLM serving. Gradient accumulation simulates training batch size but is not inference batching.

16. Interview Questions

- Why does batching improve throughput?** It amortizes overhead and exposes larger parallel operations.
- Why can it hurt latency?** Requests wait to form a batch and a larger batch takes longer to execute.
- What is dynamic batching?** Runtime grouping of independent requests under size and wait limits.
- How do you batch variable sequences?** Pad and mask, bucket by length, or use supported ragged representations.
- What is padding efficiency?** Real tokens divided by allocated padded token slots.
- How do you choose batch size?** Sweep under real shapes and concurrency while enforcing memory and p99 SLA.
- What happens above service capacity?** Queue length and latency grow; backpressure or admission control is required.
- Continuous vs dynamic batching?** Dynamic batches whole requests; continuous batching schedules individual LLM decode iterations as sequences join/finish.
- Why preserve sample IDs?** Parallel loading and retries can reorder outputs; IDs ensure correct association.
- Is batch-32 time divided by 32 single-request latency?** No; it is amortized compute time, excluding wait and request overhead.
- What is head-of-line blocking?** A slow/incompatible item delays others behind it.
- How can preprocessing be optimized?** Parallel workers, vectorization, caching, pinned buffers, and GPU preprocessing when justified.

17. Practice Tasks

- Sweep batch sizes 1–1024 and plot throughput and memory.
- Implement length buckets and measure padding saved.
- Build a small async dynamic batcher with max size and timeout.
- Debug out-of-order offline predictions using record IDs.
- Compare static batching with continuous batching for simulated LLM sequences.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Async batch inference API	Batches concurrent HTTP requests	FastAPI, asyncio, PyTorch	Synthetic/image data	Serving concurrency
Embedding pipeline	Resumable, ID-safe offline embedding	PyTorch, Parquet	Wikipedia subset	Data + inference systems
Batch-size tuner	Finds Pareto choices under SLA	Python, CUDA profiling	Model zoo	Performance experimentation

19. Quick Revision

- **Key idea:** amortize overhead and improve hardware utilization.
- **Main formula:** $\text{throughput} = B/T(B)$.
- **When to use:** independent examples and available memory/latency budget.
- **Metrics:** throughput, p99, queue delay, memory, padding efficiency.
- **Common traps:** confusing amortized time with request latency.
- **Interview one-liner:** Batch for throughput, then cap queue delay and memory to protect tail latency.

20. Final Cheat Sheet

Item	Summary
Definition	Execute multiple samples in one model call
Input/output	Compatible sample group -> aligned output group
Main steps	Queue/load, collate, execute, demultiplex
Hyperparameters	Batch size, max wait, buckets, workers
Metrics	Samples/s, p95/p99, queue time, memory, padding ratio
Pros	High utilization and throughput
Cons	Waiting, padding, memory, tail-latency cost
Best use cases	Offline scoring and throughput-oriented serving

Caching

1. Overview

Caching stores reusable results so repeated work can be skipped. AI systems cache tokenization, embeddings, retrieval results, model responses, compiled graphs, prefixes, and Transformer key/value tensors. A cache is correct only if its key includes every input and configuration element that can change the output, and if freshness/privacy requirements are enforced.

2. Intuition

Instead of solving the same expensive problem repeatedly, keep the answer in a labeled drawer. The hard part is labeling: if model version, prompt template, permissions, or preprocessing changes, an old answer may no longer be valid.

3. Prerequisites

Hashing, serialization, deterministic functions, memory/storage hierarchy, TTL and eviction, concurrency, invalidation, privacy, and latency measurement.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
Cache key	Identity of reusable computation	Determines correctness	Hash(model version + input)	What belongs in key?
Hit/miss	Entry found/not found	Governs value	Embedding reused	Hit-rate math
TTL	Time before expiry	Bounds staleness	Retrieval cache 5 min	Freshness trade-off
Eviction	Remove entries under capacity	Controls memory	LRU/LFU	Policy choice
Invalidation	Remove logically obsolete entries	Prevents stale outputs	New model version	Hardest problem
Cache-aside	App checks/fills cache	Simple and common	Redis lookup then inference	Stampede risk
Negative caching	Cache absence/errors briefly	Reduces repeated misses	Missing document	Failure poisoning
KV cache	Store attention keys/values	Avoid repeated prefix compute	Autoregressive decoding	Memory growth
Semantic cache	Reuse for similar inputs	Higher hit rate, approximate	Similar questions	Correctness threshold

5. Algorithm / Working Process

For deterministic cache-aside inference:

1. Canonicalize input safely.
2. Build a namespaced key containing model, preprocessing, prompt/schema, tenant/security scope, and input digest.
3. Look up the key.
4. On hit, validate metadata/freshness and return the result.
5. On miss, compute once, serialize, store with TTL/size limits, then return.
6. Track hit rate, latency, bytes, evictions, stale-result incidents, and compute saved.

For concurrent misses, use request coalescing/single-flight so only one worker computes a key while others await it.

6. Mathematical Foundation

Expected latency under hit rate h is approximately:

$$E[L] = h L_{\text{hit}} + (1 - h)(L_{\text{miss_lookup}} + L_{\text{compute}} + L_{\text{write}})$$

Expected compute saved per request is roughly:

$$\text{savings} = h * (C_{\text{compute}} - C_{\text{hit}})$$

For self-attention at decoding step t , recomputing all prefix projections/attention repeatedly contributes quadratic work over the sequence. A KV cache stores prior keys/values so each new token computes only its own projections and attends to cached history. Per-layer KV memory is approximately:

$$M_{\text{KV}} = 2 * B * T * H_{\text{kv}} * D_{\text{head}} * \text{bytes_per_element}$$

Multiply by number of layers. 2 represents keys and values. Grouped-query attention reduces H_{kv} .

7. Practical Implementation

```
from functools import lru_cache
import hashlib
import json

MODEL_VERSION = "sentiment-v3"

def canonical_key(text, options):
    payload = {
        "model": MODEL_VERSION,
        "text": text.strip(),
        "options": options,
    }
    raw = json.dumps(payload, sort_keys=True, separators=(",", ":")).encode()
    return hashlib.sha256(raw).hexdigest()

def expensive_predict(text, options):
    # Replace with actual model inference.
    return {"label": "positive", "score": 0.91, "key": canonical_key(text, options)}

@lru_cache(maxsize=1024)
def cached_predict(text, options_json):
```

```
options = json.loads(options_json)
return expensive_predict(text, options)

options_json = json.dumps({"language": "en"}, sort_keys=True)
first = cached_predict("great product", options_json)
second = cached_predict("great product", options_json)
print(first)
print(cached_predict.cache_info())
```

8. Code Explanation

`lru_cache` is appropriate only for a single-process demonstration with hashable arguments. Canonical JSON makes option ordering stable. A versioned key prevents serving results from an older model. Real distributed systems typically use Redis or a database, enforce TTLs and tenant boundaries, avoid storing secrets, and use single-flight or locks for hot misses.

9. Training / Evaluation

Caching is an inference/system optimization, not training. Replay a realistic request trace and measure hit rate by endpoint/tenant, hit and miss latency, p95/p99, compute/GPU time saved, memory, eviction rate, staleness, and correctness. Evaluate cold-start separately from warm-cache performance. For semantic caches, manually and automatically evaluate false-hit quality.

10. Complexity and Cost

Hash-map lookup is average $O(1)$; LRU bookkeeping is $O(1)$ with a hash map plus linked ordering. Hashing/serialization costs $O(\text{input size})$. KV cache memory grows linearly with batch and sequence length and can dominate LLM serving capacity. Distributed caches add network latency and operational cost.

11. Common Use Cases

Repeated embeddings, deterministic model responses, RAG retrieval, document parsing, feature computation, tokenization, compiled model artifacts, LLM prefix/KV caching, and deduplicating batch jobs.

12. Common Mistakes

- Omitting model/preprocessing/prompt version from the key
- Sharing entries across users with different permissions
- Caching nondeterministic or personalized results as universal
- No TTL or invalidation plan
- Cache stampedes on popular missing keys
- Caching errors for too long
- Measuring only warm-cache latency
- Unbounded in-process cache memory
- Treating semantic similarity as exact equivalence

13. Edge Cases / Limitations

High-cardinality unique requests have low hit rates. Input canonicalization can incorrectly merge meaningful differences. Stochastic generation depends on seed and sampling parameters. User deletion/privacy rules may require targeted eviction. KV caches grow large and may fragment GPU memory; long-context eviction can hurt quality.

14. Variations

Variation	What changes / when to use	Importance
Exact result cache	Byte/canonical equality	Essential baseline
Semantic cache	Similarity threshold allows approximate reuse	RAG/LLM projects
Feature/embedding cache	Stores intermediate vectors	Common ML systems
Prefix cache	Reuses shared prompt-prefix computation	LLM serving
KV cache	Reuses per-layer attention K/V	Core autoregressive inference
Multi-level cache	GPU/RAM/distributed/disk hierarchy	Systems research

15. Related Topics

RAG caches can sit at embedding, retrieval, reranking, or generation layers. Batch inference handles new work efficiently; caching removes repeated work. Quantization reduces KV-cache bytes. Speculative decoding and prefix caching accelerate LLM generation differently. Feature stores provide governed reusable features rather than merely transient caching.

16. Interview Questions

1. **What should a model cache key contain?** Every output-affecting version, input, option, and security/tenant scope.
2. **How do you calculate expected cache latency?** Weight hit and miss paths by the hit rate.
3. **What is cache invalidation?** Removing entries that are no longer semantically valid.
4. **What is a cache stampede?** Many workers recompute the same hot key after a miss/expiry.
5. **How do you prevent stampedes?** Single-flight, per-key locking, jittered TTLs, or stale-while-revalidate.
6. **What is semantic caching?** Approximate reuse when embedding similarity exceeds a threshold.
7. **Why is semantic caching risky?** Similar wording may require different answers; false hits silently reduce correctness.
8. **What is an LLM KV cache?** Stored attention keys/values for earlier tokens at every layer.
9. **KV-cache trade-off?** Lower decoding compute at substantial memory cost growing with sequence and batch.
10. **Why version cache keys?** Deployments and preprocessing changes otherwise return obsolete outputs.
11. **LRU vs LFU?** LRU evicts least recently used; LFU favors frequently reused entries and adapts differently to workload shifts.
12. **How do you test a cache?** Cold/warm paths, invalidation, concurrency, tenant isolation, eviction, failures, and realistic trace replay.

17. Practice Tasks

- Add a versioned LRU cache to an embedding function.
- Implement TTL and per-key single-flight for an async service.
- Compute KV-cache memory for several LLM configurations.
- Debug cross-user leakage caused by a missing tenant key field.
- Evaluate semantic-cache thresholds with false-hit analysis.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Cached RAG API	Caches embeddings/retrieval with versioning	FastAPI, Redis, vector DB	BEIR subset	Production RAG design
Semantic cache evaluator	Measures savings vs incorrect reuse	Sentence Transformers	FAQ dataset	Quality/cost trade-off
KV memory planner	Predicts serving capacity	Python, Transformers configs	Open model configs	LLM systems understanding

19. Quick Revision

- **Key idea:** avoid recomputing safely reusable work.
- **Main formula:** $E[L] = h L_{hit} + (1-h)L_{miss}$.
- **When to use:** repeated expensive deterministic or acceptably approximate queries.
- **Metrics:** hit rate, hit/miss p99, compute saved, stale/false hits, bytes.
- **Common traps:** incomplete keys, leaks, stampedes, unbounded memory.
- **Interview one-liner:** A cache is a correctness mechanism first and a latency optimization second.

20. Final Cheat Sheet

Item	Summary
Definition	Store reusable computation results
Input/output	Complete versioned key -> cached value or miss
Main steps	Canonicalize, lookup, validate, compute, store, invalidate
Hyperparameters	Capacity, TTL, eviction, similarity threshold
Metrics	Hit rate, p99, saved compute, stale/false-hit rate
Pros	Large latency and cost reduction on repeated work
Cons	Staleness, privacy, memory, invalidation complexity
Best use cases	Repeated embeddings/results, prefixes, LLM decoding

Latency vs Throughput

1. Overview

Latency is the time one request takes; throughput is work completed per unit time. Efficient AI systems must optimize both under quality, cost, memory, and reliability constraints. They often conflict: batching raises throughput but adds queueing and service time. Production decisions use latency distributions and sustainable throughput, not a single average.

2. Intuition

A checkout lane can minimize one shopper's time or process many shoppers per hour. Opening batches and keeping every cashier busy may improve total flow, yet an individual can wait longer. The same tension appears in inference queues and accelerators.

3. Prerequisites

Probability percentiles, timers, warm-up, concurrency, queues, batch inference, CPU/GPU synchronization, service-level objectives (SLOs), and basic capacity planning.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Interview angle
End-to-end latency	Full client-observed time	Represents experience	Network + queue + model	What exactly was timed?
Service time	Time actively processing	Capacity component	GPU inference 8 ms	Distinguish queueing
p50/p95/p99	Latency distribution percentiles	Tails drive SLOs	p99 < 200 ms	Why average misleads
Throughput	Completed units/time	Measures capacity	2,000 samples/s	Requests vs tokens
Concurrency	In-flight requests	Helps saturate resources	32 workers	Not same as parallelism
Utilization	Busy resource fraction	Efficiency/saturation signal	GPU 75% active	High utilization risk
Cold start	First-load/compile latency	Important for autoscaling	Model load 4 s	Separate benchmark
Goodput	Useful work meeting constraints	Better than raw throughput	Correct requests under SLO	Quality-aware systems metric

5. Algorithm / Working Process

1. Define the request unit and latency boundary.

2. Define workload distributions: shapes, sequence lengths, arrivals, cache hit rate, and concurrency.
3. Warm up model, kernels, allocators, and caches.
4. Generate load at controlled offered rates.
5. Record per-request timestamps and outcomes.
6. Report p50/p95/p99/max, throughput, goodput, utilization, queue time, and errors.
7. Increase load until SLO violations or instability reveal sustainable capacity.
8. Tune batch size, concurrency, replicas, precision, caching, and admission control.

6. Mathematical Foundation

```
latency_i = completion_time_i - arrival_time_i
throughput = completed_requests / observation_time
goodput = valid_requests_meeting_SLO / observation_time
```

Little's Law for a stable system:

```
average_concurrency = throughput * average_latency
```

For a single server with arrival rate λ and mean service rate μ , utilization is:

```
rho = lambda / mu, requiring rho < 1 for stability
```

In an idealized M/M/1 queue:

```
E[response_time] = 1 / (mu - lambda)
```

This shows latency grows sharply near saturation. Amdahl's Law bounds optimization:

```
speedup = 1 / ((1 - p) + p/s)
```

If only fraction p is accelerated by factor s , the rest limits end-to-end gain.

7. Practical Implementation

```
from statistics import mean
from time import perf_counter
import numpy as np
import torch
from torch import nn
```

```

device = "cuda" if torch.cuda.is_available() else "cpu"
model = nn.Sequential(nn.Linear(1024, 2048), nn.ReLU(), nn.Linear(2048, 64))
model = model.to(device).eval()
x = torch.randn(32, 1024, device=device)

def sync():
    if device == "cuda":
        torch.cuda.synchronize()

with torch.inference_mode():
    for _ in range(30):
        model(x)
        sync()

    latencies_ms = []
    wall_start = perf_counter()
    for _ in range(200):
        start = perf_counter()
        model(x)
        sync()
        latencies_ms.append((perf_counter() - start) * 1000)
    wall_time = perf_counter() - wall_start

samples = 200 * len(x)
print({
    "mean_ms_per_batch": mean(latencies_ms),
    "p50_ms": float(np.percentile(latencies_ms, 50)),
    "p95_ms": float(np.percentile(latencies_ms, 95)),
    "p99_ms": float(np.percentile(latencies_ms, 99)),
    "samples_per_second": samples / wall_time,
})

```

8. Code Explanation

Warm-up excludes initialization and kernel-selection effects from steady-state results. CUDA synchronization makes each recorded interval include completed GPU work; without it, the loop mostly times enqueue operations. Per-batch percentile latency and sample throughput are labeled separately. A production load test should record request-level queue/network time and avoid synchronizing every operation if doing so changes normal execution.

9. Training / Evaluation

This is primarily inference evaluation. Fix model quality first; a fast wrong model has no goodput. Use realistic open-loop arrivals to reveal queue buildup and closed-loop tests to study client-observed behavior. Run long enough to capture autoscaling, garbage collection, thermal effects, cache shifts, and tail events. Compare equal hardware/power and identical preprocessing.

10. Complexity and Cost

Higher throughput can reduce cost per prediction but require larger batches and more memory. More replicas lower queueing and improve availability but cost more. Tail latency may require headroom, so operating at

theoretical maximum utilization is unsafe. Cold-start optimization affects scale-to-zero economics.

11. Common Use Cases

Selecting serving batch size, capacity planning, comparing inference engines, autoscaling, setting SLOs, optimizing real-time detection, and benchmarking LLM prefill/decode performance.

12. Common Mistakes

- Reporting averages without percentiles
- Timing only model compute while claiming end-to-end latency
- Comparing different batch sizes as if latency units were identical
- Forgetting CUDA synchronization
- Measuring only batch-1 with no concurrent load
- Driving unstable offered load and calling completed rate capacity
- Ignoring warm-up/cold-start distinction
- Reporting tokens/s without prompt/output lengths or batch/concurrency
- Optimizing throughput beyond the p99 SLO

13. Edge Cases / Limitations

Bursty arrivals violate simple stationary queue assumptions. Heavy-tailed sequence lengths cause head-of-line blocking. Cached and uncached paths form different latency populations. GPU power/thermal throttling changes long-run results. Coordinated omission in load generators can hide bad tail latency when clients wait before sending new requests.

14. Variations

Variation	What changes / when to use	Importance
Online latency benchmark	Real request path and concurrency	Essential production skill
Kernel/model microbenchmark	Isolates compute bottleneck	Debugging only
Offline throughput benchmark	Saturates finite dataset pipeline	Batch workloads
Goodput benchmark	Counts only correct, SLO-compliant work	Advanced systems interviews
LLM TTFT/TPOT	Separates prefill and decode experience	Essential LLM serving
Energy efficiency	Work per joule	Edge/data-center cost

15. Related Topics

Batch inference moves along the latency-throughput curve. Caching lowers both compute and latency on hits. Quantization/TensorRT reduce service time. Queueing, backpressure, autoscaling, load shedding, and SLO design determine system behavior. For LLMs, time to first token (TTFT), time per output token (TPOT), and tokens/s are distinct.

16. Interview Questions

1. **Latency vs throughput?** Latency measures time per request; throughput measures completed work per time.
2. **Why report p99?** Rare slow requests matter to user experience and distributed fan-out systems.
3. **How does batching affect both?** It typically raises throughput but adds queueing and larger batch service time.
4. **What is sustainable throughput?** Highest arrival rate maintained without unbounded queues, errors, or SLO failure.
5. **What does Little's Law say?** Average in-flight work equals throughput times average response time in a stable system.
6. **Why does latency explode near saturation?** Small service variability creates queues when little spare capacity remains.
7. **What is goodput?** Correct, useful work completed within required constraints per unit time.
8. **How do you benchmark CUDA correctly?** Warm up and use synchronization or CUDA events around the intended boundary.
9. **What is coordinated omission?** A load generator sends less work while blocked, failing to record latency users would see under continued arrivals.
10. **What is Amdahl's Law's lesson?** Accelerating one component has limited end-to-end value when other components remain.
11. **What are TTFT and TPOT?** Time to first generated token and time per subsequent output token.
12. **Would you maximize GPU utilization?** Not blindly; reserve headroom for bursts and p99 SLOs.

17. Practice Tasks

- Build batch-size latency/throughput curves.
- Add request-level p50/p95/p99 instrumentation to an API.
- Simulate an M/M/1 queue and observe latency near saturation.
- Find a benchmark bug caused by missing CUDA synchronization.
- Compare model-only latency with end-to-end preprocessing/network latency.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Inference load lab	Generates realistic loads and percentile reports	FastAPI, Locust/k6, Prometheus	Any model	Production observability
SLO-aware batcher	Tunes batch window under p99 target	asyncio, PyTorch	Synthetic trace	Queueing + ML systems
LLM serving dashboard	Tracks TTFT, TPOT, tokens/s, KV memory	vLLM/TGI, Grafana	Prompt trace	LLMOps relevance

19. Quick Revision

- **Key idea:** optimize user delay and system capacity together.
- **Main formulas:** $\text{throughput} = N / \text{time}$, $\text{concurrency} = \text{throughput} * \text{latency}$.
- **When to use:** every deployment benchmark and capacity plan.

- **Metrics:** p50/p95/p99, goodput, queue time, utilization, error rate.
- **Common traps:** averages, async timing, unrealistic workload.
- **Interview one-liner:** Maximum throughput is not production capacity unless tail latency and stability remain within SLO.

20. Final Cheat Sheet

Item	Summary
Definition	Request time vs completed work rate
Input/output	Workload and system -> latency distribution/capacity curve
Main steps	Define boundary, warm up, load, record, analyze, tune
Hyperparameters	Batch, concurrency, replicas, queue limits, precision
Metrics	p50/p95/p99, requests/s or tokens/s, goodput, utilization
Pros	Exposes true performance trade-offs
Cons	Results depend strongly on workload and measurement design
Best use cases	Serving design, capacity planning, engine comparison

Low-Rank Adaptation

1. Overview

Low-Rank Adaptation (LoRA) is a parameter-efficient fine-tuning method. It freezes a pretrained weight matrix and learns a low-rank update instead of updating every parameter. This greatly reduces trainable parameters and optimizer memory, enabling many task-specific adapters to share one base model. LoRA is common in LLMs, vision Transformers, diffusion models, and multimodal systems.

LoRA primarily reduces fine-tuning cost and checkpoint size. It does not reduce base-model inference compute by itself; an adapter can be merged into the base weights to avoid extra inference operations.

2. Intuition

A pretrained model already knows general patterns. A new task usually needs a small directional adjustment, not a complete rewrite of every weight. LoRA expresses that adjustment using two thin matrices, like describing a large edit through a small set of reusable directions.

3. Prerequisites

Matrix multiplication, rank, singular value decomposition intuition, Transformer linear projections, backpropagation, fine-tuning, optimizer state memory, and PyTorch modules.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Common interview angle
Frozen base	Original weights receive no gradients	Saves optimizer/gradient memory	Freeze LLM attention weights	Does LoRA reduce model load memory?
Low-rank update	$\Delta W = BA$ with rank r	Few trainable values	Rank 8 update to 4096x4096 matrix	Parameter count
Rank r	Bottleneck dimension	Capacity/cost control	$r=4,8,16,64$	How to choose rank
Scaling α/r	Controls adapter magnitude	Stabilizes rank comparisons	$\alpha=16, r=8$	Why scale?
Target modules	Layers receiving adapters	Determines expressiveness/cost	Q/V or all linear layers	Q/V-only vs broader targeting
Adapter dropout	Regularizes adapter path	Helps small/noisy data	Dropout 0.05	Training only
Merge/unmerge	Add update into base weight	Removes runtime adapter overhead	$W \leftarrow W + sBA$	Multi-adapter implications
PEFT	Family of efficient tuning methods	Puts LoRA in context	Prefix/prompt/adapters	LoRA vs prompt tuning

5. Algorithm / Working Process

1. Load a pretrained model and freeze base parameters.
2. Select target linear layers such as attention query/value and sometimes key/output/MLP projections.
3. For each selected weight W , create trainable matrices A and B of rank r .
4. Compute the normal projection plus a scaled low-rank update.
5. Train only adapter parameters (and optionally biases/norms/task head).
6. Save a small adapter checkpoint plus base-model identifier.
7. At inference, load the base and adapter; optionally merge the update into W .

Input/output and task loss remain those of the original model. Only the parameterization of fine-tuning changes.

6. Mathematical Foundation

For base weight W_0 in $\mathbb{R}^{(d_{out} \times d_{in})}$:

$$W = W_0 + \Delta W$$

$$\Delta W = (\alpha / r) B A$$

$$A \in \mathbb{R}^{(r \times d_{in})}, B \in \mathbb{R}^{(d_{out} \times r)}$$

$$y = W_0 x + (\alpha / r) B(Ax)$$

The base has $d_{out} \times d_{in}$ parameters. LoRA trains:

$$N_{LoRA} = r(d_{in} + d_{out})$$

$$\text{ratio} = r(d_{in} + d_{out}) / (d_{in} d_{out})$$

For a square 4096×4096 matrix and $r=8$, LoRA trains 65,536 rather than 16,777,216 parameters: about 0.39% for that matrix.

With task loss L , gradients update only A, B :

```
A <- A - eta * dL/dA
B <- B - eta * dL/dB
W_0 remains fixed
```

Common initialization uses random A and zero B , so $\Delta W=0$ initially and the starting function exactly matches the base model.

7. Practical Implementation

```
import math
import torch
from torch import nn

class LoRALinear(nn.Module):
    def __init__(self, base: nn.Linear, rank=8, alpha=16.0, dropout=0.05):
        super().__init__()
        self.base = base
        self.rank = rank
        self.scale = alpha / rank
        self.dropout = nn.Dropout(dropout)

        for parameter in self.base.parameters():
            parameter.requires_grad = False

        self.A = nn.Parameter(torch.empty(rank, base.in_features))
        self.B = nn.Parameter(torch.zeros(base.out_features, rank))
        nn.init.kaiming_uniform_(self.A, a=math.sqrt(5))

    def forward(self, x):
        base_output = self.base(x)
        update = (self.dropout(x) @ self.A.T) @ self.B.T
```

```

        return base_output + self.scale * update

layer = LoRALinear(nn.Linear(1024, 1024), rank=8)
trainable = sum(p.numel() for p in layer.parameters() if p.requires_grad)
total = sum(p.numel() for p in layer.parameters())
print(f"trainable={trainable:,} / total={total:,} ({trainable/total:.2%})")

x = torch.randn(4, 1024)
loss = layer(x).square().mean()
loss.backward()
assert layer.base.weight.grad is None
assert layer.A.grad is not None and layer.B.grad is not None

```

8. Code Explanation

The original `Linear` layer remains frozen. The adapter follows $x \rightarrow A \rightarrow B$, creating a rank-at-most- r update. Zero-initialized `B` makes the initial adapter output zero. Only `A` and `B` receive gradients, as verified by assertions. In real LLM projects, Hugging Face PEFT injects and saves adapters safely rather than requiring a custom module.

9. Training / Evaluation

Prepare instruction/task data with train/validation/test splits that avoid document, user, or temporal leakage. Use the base tokenizer and correct label masking—for causal language modeling, prompt tokens may be excluded from loss. Evaluate task quality, general-capability regression, hallucination/safety, adapter size, peak training memory, tokens/s, and merged/unmerged latency. Tune rank, alpha, target modules, dropout, learning rate, epochs, context length, and data mixture.

10. Complexity and Cost

For a target linear layer, extra forward work is approximately $O(r(d_{in}+d_{out}))$ per token instead of a full additional $O(d_{in}*d_{out})$. Trainable gradients and optimizer states scale with adapter size, but frozen base weights and activations needed for backpropagation still consume memory. Quantizing the base (QLoRA) reduces base-weight memory further. Many adapters can share one loaded base.

11. Common Use Cases

Domain adaptation of LLMs, personalized assistants, instruction/style tuning, diffusion style/concept adapters, multilingual adaptation, federated/local fine-tuning, and serving many customer-specific variants.

12. Common Mistakes

- Claiming LoRA makes the base model small or inherently faster
- Training the base accidentally because parameters were not frozen
- Targeting wrong module names
- Saving only adapters without base model/version/tokenizer metadata
- Using too high rank on tiny data and overfitting
- Applying task loss to padded or prompt tokens incorrectly
- Merging adapters repeatedly into already merged weights

- Comparing against full fine-tuning with different data/hyperparameters
- Ignoring license/security risks in third-party adapters

13. Edge Cases / Limitations

Some tasks require high-rank or nonlinear/full-model changes. LoRA can underperform full fine-tuning under major domain shift. Multiple active adapters add scheduling and memory complexity. Quantized merging may require dequantization/requantization. Catastrophic behavior changes can still occur despite few trained parameters.

14. Variations

Variation	What changes / when to use	Importance
QLoRA	Frozen base stored in 4-bit, adapter trained higher precision	Essential LLM topic
AdaLoRA	Allocates rank adaptively across layers	Research interviews
DoRA	Separates weight magnitude and directional adaptation	Advanced/research
LoRA+	Uses different learning rates for A and B	Research optimization
Multi-adapter serving	Switch/compose adapters over one base	Production LLM systems
LoRA for diffusion	Targets attention projections in image generators	GenAI projects

15. Related Topics

LoRA vs QLoRA: QLoRA quantizes the frozen base while retaining LoRA adapters. Full fine-tuning updates all weights and costs more but offers maximum flexibility. Prompt/prefix tuning trains input-like vectors rather than weight updates. Distillation creates a standalone student; LoRA still depends on the base model. SVD motivates low-rank structure but LoRA learns the factors directly.

16. Interview Questions

1. **What is LoRA?** A low-rank trainable update added to frozen pretrained weights.
2. **Why can low rank work?** Task adaptation often lies in a much lower-dimensional subspace than the full parameter space.
3. **How many parameters does a LoRA layer add?** $r(d_{in}+d_{out})$.
4. **Why initialize B to zero?** The adapter initially changes nothing, preserving the pretrained function.
5. **What does alpha do?** Scales update magnitude, commonly through α/r .
6. **Does LoRA reduce inference FLOPs?** Not inherently; unmerged adapters add small operations, while merging restores the original layer shape.
7. **LoRA vs QLoRA?** QLoRA stores the frozen base in typically 4-bit form during adapter training.
8. **Which Transformer layers are targeted?** Often query/value projections; broader targets can improve quality at higher cost.
9. **Can adapters be merged?** Yes, add the scaled BA update to the base weight when dtype/serving requirements allow.

10. **What memory does LoRA save?** Gradients and optimizer states for frozen parameters; it does not eliminate base weights/activations.
11. **How do you choose rank?** Validation sweep under memory/quality constraints; harder shifts often need more capacity.
12. **Main deployment benefit?** Many small task adapters can share one base model.

17. Practice Tasks

- Implement **LoRALinear** and verify only adapter gradients exist.
- Fine-tune a text classifier with ranks 2, 8, and 32.
- Compare full fine-tuning, LoRA, and QLoRA memory/quality.
- Debug adapter loading against a mismatched base revision.
- Merge an adapter and numerically verify outputs before/after.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain LLM adapter	Adapts an open LLM to a specialist QA domain	Transformers, PEFT, TRL	Medical/legal public QA	Modern fine-tuning skill
Multi-adapter router	Serves tenant-specific adapters over one base	PEFT, FastAPI, vLLM	Synthetic tenant styles	Systems + personalization
LoRA experiment study	Maps rank/targets/data to quality and memory	PyTorch, MLflow	GLUE/AG News	Research rigor

19. Quick Revision

- **Key idea:** learn $\Delta W = BA$ while freezing W_0 .
- **Main formula:** $y = W_0x + (\alpha/r)B(Ax)$.
- **When to use:** pretrained model is strong but full fine-tuning is too expensive.
- **Metrics:** task quality, trainable count, peak memory, training speed, latency.
- **Common traps:** wrong targets, unfrozen base, base-version mismatch.
- **Interview one-liner:** LoRA compresses the adaptation, not the base model.

20. Final Cheat Sheet

Item	Summary
Definition	Low-rank parameter-efficient weight update
Input/output	Pretrained model + task data -> small adapter
Main steps	Freeze, inject A/B, train, save, optionally merge
Hyperparameters	Rank, alpha, dropout, target modules, LR
Metrics	Quality, trainable params, memory, train time, latency
Pros	Tiny checkpoints, lower optimizer memory, shareable base

Item	Summary
Cons	Base still required; capacity/target selection matter
Best use cases	Many adaptations of large pretrained models

Sparse Models

1. Overview

Sparse models perform computation or store parameters with many absent/zero elements. Sparsity may exist in weights, activations, attention connections, or expert routing. It can reduce memory and compute, but useful acceleration depends on sparsity pattern, density, shapes, formats, and hardware kernels. Sparse models include pruned networks, sparse attention, sparse mixture-of-experts (MoE), and classical sparse linear models.

2. Intuition

A dense model checks every possible connection. A sparse model follows only selected roads. If roads are irregular, reading a map of indices may cost more than simply traversing a dense grid; regular blocks or hardware-supported patterns are easier to accelerate.

3. Prerequisites

Sparse matrices, CSR/CSC/COO formats, pruning, regularization, attention, GPU memory access, load balancing, and benchmarking.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Common interview angle
Weight sparsity	Many zero parameters	Potential storage/compute savings	90% pruned MLP	Sparse format overhead
Activation sparsity	Runtime activations are zero	Input-dependent savings	ReLU outputs	Hardware exploitation
Structured sparsity	Whole blocks/channels absent	Regular fast execution	Block sparse matrix	Speed vs flexibility
Semi-structured N:M	N values kept per M group	Matches accelerators	2:4 sparsity	Hardware constraints
Sparse attention	Restrict token connections	Scales long contexts	Local window + globals	Complexity reduction

Subtopic	Meaning	Why it matters	Example	Common interview angle
Mixture of Experts	Only top-k experts run per token	Huge capacity, bounded active compute	Top-2 of 64 experts	Load balancing/routing
Sparse storage	Values plus indices	Saves space above break-even sparsity	CSR	When sparse is larger
Dynamic sparsity	Pattern changes with data/training	More adaptive, harder to optimize	Conditional routing	Static vs dynamic

5. Algorithm / Working Process

The generic lifecycle is:

1. Choose what becomes sparse: weights, connections, attention, activations, or experts.
2. Choose a pattern supported by target hardware/runtime.
3. Produce sparsity through pruning, regularization, routing, or architecture design.
4. Fine-tune while enforcing the pattern.
5. Encode values and metadata in a suitable sparse or smaller dense representation.
6. Execute with compatible sparse kernels.
7. Measure task quality, effective density, load balance, memory, and end-to-end speed.

MoE inference additionally computes router scores, selects top-k experts, dispatches tokens, executes expert MLPs, and combines weighted outputs.

6. Mathematical Foundation

Sparse matrix-vector multiplication:

```
y_i = sum_{j in nonzero_indices(row i)} A_ij x_j
cost approximately O(nnz(A))
```

where `nnz` is number of nonzeros, versus dense $O(mn)$. Index decoding and irregular access add constants.

For sparse attention with each token attending to at most `w` positions:

```
dense attention: O(T^2 d)
sparse attention: O(T w d), where w << T
```

For top-k MoE routing:

```
p(e|x) = softmax(router(x))
TopK = k experts with largest p(e|x)
```

$$y = \sum_{e \in \text{TopK}} p(e|x) \text{ Expert}_e(x)$$

A load-balancing auxiliary loss encourages uniform expert usage; conceptually:

$$L_{\text{total}} = L_{\text{task}} + \lambda L_{\text{balance}}$$

7. Practical Implementation

```
import torch

# Construct a sparse COO matrix and compare with its dense equivalent.
indices = torch.tensor([[0, 0, 1, 2],
                        [0, 2, 1, 3]])
values = torch.tensor([2.0, -1.0, 3.0, 4.0])
A_sparse = torch.sparse_coo_tensor(indices, values, size=(3, 4)).coalesce()
x = torch.tensor([[1.0], [2.0], [3.0], [4.0]])

y_sparse = torch.sparse.mm(A_sparse, x)
y_dense = A_sparse.to_dense() @ x

torch.testing.assert_close(y_sparse, y_dense)
print("output:", y_sparse.squeeze().tolist())
print("density:", A_sparse._nnz() / A_sparse.numel())
```

8. Code Explanation

COO stores nonzero coordinates and values rather than every zero. `coalesce` merges duplicate coordinates and normalizes representation. Sparse and dense multiplication must agree numerically. This tiny example demonstrates semantics, not speed: small sparse operations often lose to optimized dense kernels due to metadata and launch overhead.

9. Training / Evaluation

Evaluate the dense baseline, sparse mathematical model, and actual sparse runtime separately. Track task metrics, nonzero count, pattern compliance, index overhead, active parameters/FLOPs, latency, throughput, and memory. For MoE, also monitor tokens per expert, overflow/dropped tokens, router entropy, communication time, and expert capacity factor. For sparse attention, test tasks requiring long-range connections.

10. Complexity and Cost

Ideal sparse compute scales with `nnz`, but real performance depends on regularity and arithmetic intensity. CSR stores approximately `nnz` values, `nnz` column indices, and `rows+1` row pointers. MoE reduces active compute relative to total parameters but may incur all-to-all communication. Sparse attention lowers quadratic attention cost, though projections/MLPs may still dominate.

11. Common Use Cases

Long-context Transformers, conditional-compute LLMs, recommender embeddings/features, graph neural networks, pruned edge models, scientific matrices, and sparse linear models with interpretable selected features.

12. Common Mistakes

- Treating zeros as compressed storage automatically
- Ignoring index metadata and irregular memory access
- Benchmarking a sparse kernel at only one shape/density
- Confusing total MoE parameters with active parameters per token
- Letting experts become overloaded or unused
- Using sparse attention that removes task-critical long-range links
- Claiming FLOP reduction as proportional wall-clock speedup
- Converting dense to sparse on every inference call

13. Edge Cases / Limitations

Sparse kernels can underperform dense kernels at moderate density or small shapes. Random sparsity is difficult for SIMD/tensor cores. MoE communication and load imbalance dominate at scale. Sparse attention patterns can lose global context. Some operators and autograd paths have limited sparse support.

14. Variations

Variation	What changes / when to use	Importance
Unstructured weights	Highest flexibility, specialized kernels needed	Pruning research
Block sparsity	Fixed dense blocks	GPU-friendly projects
N:M sparsity	Local pattern such as 2:4	Hardware interviews
Local/global attention	Sparse token graph	Long-context NLP
Top-k MoE	Sparse expert activation	Essential modern LLM topic
L1-sparse classical models	Feature selection via regularization	Placement fundamentals

15. Related Topics

Pruning creates weight sparsity; sparse modeling includes broader architectural sparsity. Quantization reduces bits per stored nonzero and can combine with sparsity. MoE increases total capacity while limiting active compute. Custom kernels are often necessary to realize sparse speedups. Low-rank matrices are compact but generally dense and differ from sparse matrices.

16. Interview Questions

1. **What is a sparse model?** A model where many weights, activations, connections, or components are inactive/absent.

2. **Why can sparse be slower than dense?** Indices, irregular access, low utilization, conversions, and mature dense kernels.
3. **Structured vs unstructured sparsity?** Structured follows regular groups; unstructured removes arbitrary values.
4. **What is CSR?** Row pointers plus column indices and nonzero values.
5. **What is N:M sparsity?** Exactly N retained values within each group of M.
6. **How does sparse attention reduce complexity?** Each token attends to w rather than all T tokens, reducing $O(T^2d)$ to $O(Twd)$.
7. **What is sparse MoE?** A router activates only a few expert networks for each token.
8. **Why use load-balancing loss in MoE?** Prevent expert collapse and capacity overflow.
9. **Total vs active MoE parameters?** Total counts all experts; active counts only selected experts per token.
10. **When does sparse storage save memory?** When zero fraction is high enough to offset index/pointer metadata.
11. **Static vs dynamic sparsity?** Static pattern is fixed and easier to compile; dynamic pattern adapts to input but complicates execution.
12. **How do you prove sparse acceleration?** Benchmark the target sparse format/kernel end-to-end at realistic shapes and density.

17. Practice Tasks

- Compare COO/CSR/dense memory for several densities.
- Benchmark sparse vs dense matmul across matrix sizes.
- Implement a local-window attention mask and verify connectivity.
- Diagnose MoE expert imbalance from routing statistics.
- Enforce and validate a 2:4 mask on weight blocks.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Sparse-kernel break-even lab	Finds density/shape crossover points	PyTorch, SciPy, CUDA profiling	Synthetic matrices	Honest systems benchmarking
Long-document classifier	Compares dense and sparse attention	Transformers, Longformer	arXiv/IMDB long docs	Algorithm + efficiency
Mini-MoE	Implements routing and load analysis	PyTorch	WikiText subset	Modern architecture depth

19. Quick Revision

- **Key idea:** compute/store only selected connections or components.
- **Main formula:** sparse matmul ideally costs $O(nnz)$.
- **When to use:** high exploitable sparsity and matching kernels/hardware.
- **Metrics:** quality, density, active FLOPs, index bytes, balance, latency.
- **Common traps:** zeros/FLOPs do not guarantee acceleration.
- **Interview one-liner:** Sparsity is an algorithmic property; speed is a format-and-kernel property.

20. Final Cheat Sheet

Item	Summary
Definition	Models with many inactive weights/connections/components
Input/output	Dense/sparse inputs -> output using selected computation
Main steps	Choose pattern, train/prune, encode, run sparse kernel, validate
Hyperparameters	Density, block/N:M pattern, top-k, window, balance weight
Metrics	Quality, nnz, active FLOPs, memory, latency, load balance
Pros	Potential capacity, memory, and compute efficiency
Cons	Metadata, irregularity, kernel/communication complexity
Best use cases	High supported sparsity, long attention, conditional experts

Edge Deployment

1. Overview

Edge deployment runs inference near data sources—phones, browsers, cameras, robots, vehicles, gateways, or microcontrollers—rather than relying entirely on a remote cloud. It provides low network-independent latency, privacy, offline operation, and reduced bandwidth, but must satisfy strict compute, memory, power, thermal, package-size, and operator constraints.

2. Intuition

Cloud inference is asking a remote expert; edge inference carries a compact expert in your pocket. The local expert responds without a connection and keeps raw data nearby, but has a smaller brain and battery.

3. Prerequisites

Model export, quantization, pruning/distillation, hardware accelerators, preprocessing, profiling, mobile/embedded runtimes, privacy/security, and application packaging.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Common interview angle
Target-first design	Optimize for exact device/runtime	Avoids unsupported models	Android NNAPI vs MCU	Why desktop benchmark is insufficient
On-device runtime	Executes optimized artifact	Determines operator support	TFLite, Core ML, ORT Mobile	Runtime choice

Subtopic	Meaning	Why it matters	Example	Common interview angle
Hardware delegate	Maps ops to accelerator	Main speed/energy gain	GPU/NPU/DSP delegate	Partial delegation
Footprint	Binary + model + working memory	Device constraint	20 MB app model	Weights vs peak RAM
Power/thermal	Energy and heat under sustained load	Controls reliability/performance	Camera runs continuously	Throttling
Offline/privacy	Raw data stays local	Product/security advantage	Wake-word detection	Model outputs may still be sensitive
Hybrid edge-cloud	Local fast path plus cloud fallback	Balances quality and availability	Local intent, cloud complex query	Routing policy
OTA updates	Safely update model artifact	Fixes drift/security	Signed model rollout	Rollback/versioning

5. Algorithm / Working Process

1. Specify target device, OS/runtime, accuracy, p95 latency, RAM, size, energy, and offline requirements.
2. Choose an edge-friendly architecture and preprocessing pipeline.
3. Train and establish a representative baseline.
4. Compress using distillation, structured pruning, quantization, or smaller input resolution.
5. Export to the target runtime and replace unsupported operators.
6. Validate numerical/task parity on representative and safety-critical samples.
7. Profile end-to-end on physical devices under sustained load.
8. Package preprocessing/postprocessing, versioning, signatures, telemetry, fallback, and rollback.

Inference input may be sensor/audio/image/text data; processing should avoid needless format conversions; output is a compact local prediction or decision.

6. Mathematical Foundation

Peak working memory is more than model size:

$$M_{\text{peak}} \text{ approximately } M_{\text{weights}} + M_{\text{activations}} + M_{\text{workspace}} + M_{\text{runtime}} + M_{\text{input_output}}$$

Energy per inference can be measured as:

$$E_{\text{inference}} = \int_{\text{start_to_end}} P(t) dt$$

Average device duty-cycle power for inference rate r is roughly:

$$P_{\text{average}} \text{ approximately } r * E_{\text{inference}} + P_{\text{idle}}$$

For hybrid routing, expected latency is:

$$E[L] = p_{\text{edge}} L_{\text{edge}} + (1 - p_{\text{edge}})(L_{\text{route}} + L_{\text{network}} + L_{\text{cloud}})$$

Compression must satisfy a constrained objective:

```
maximize quality subject to latency <= L_max,
memory <= M_max, energy <= E_max, size <= S_max
```

7. Practical Implementation

Export a compact PyTorch model to ONNX and create a mobile-friendly optimized artifact with ONNX Runtime tools:

```
import time
import numpy as np
import torch
from torch import nn
import onnxruntime as ort

class TinyEdgeModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.net = nn.Sequential(
            nn.Conv2d(3, 16, 3, stride=2, padding=1),
            nn.ReLU(),
            nn.AdaptiveAvgPool2d(1),
            nn.Flatten(),
            nn.Linear(16, 4),
        )

    def forward(self, x):
        return self.net(x)

model = TinyEdgeModel().eval()
sample = torch.randn(1, 3, 96, 96)
torch.onnx.export(
```

```

    model,
    (sample,),
    "tiny_edge.onnx",
    input_names=["image"],
    output_names=["logits"],
    opset_version=17,
)

session = ort.InferenceSession("tiny_edge.onnx", providers=
["CPUExecutionProvider"])
image = np.random.randn(1, 3, 96, 96).astype(np.float32)
for _ in range(20):
    session.run(None, {"image": image})

times_ms = []
for _ in range(100):
    start = time.perf_counter()
    session.run(None, {"image": image})
    times_ms.append((time.perf_counter() - start) * 1000)

print("p50 ms:", float(np.percentile(times_ms, 50)))
print("p95 ms:", float(np.percentile(times_ms, 95)))

```

8. Code Explanation

The model uses a small input and global pooling rather than a large fully connected feature map. ONNX export separates training from deployment. The session is created once; repeated creation would distort latency and waste resources. This desktop CPU benchmark is only a development check—the final decision requires the exact phone/board/runtime and end-to-end camera or sensor pipeline.

9. Training / Evaluation

Training data must reflect edge sensors: compression artifacts, microphones, orientation, lighting, motion, device variation, and environmental noise. Split by user/device/location/time when random splitting would leak context. Evaluate quality by subgroup, p50/p95 latency, peak RAM, package size, battery/energy, sustained thermal behavior, crash rate, unsupported-op fallback, and offline robustness.

10. Complexity and Cost

Cloud cost and network bandwidth fall, while device engineering, testing matrix, and update complexity rise. INT8 often reduces weights by about 4x from FP32, but peak RAM includes activations/workspace. Lower resolution reduces many vision operations quadratically with spatial dimension. Cold startup and model mapping can dominate one-shot tasks.

11. Common Use Cases

Wake-word detection, keyboard prediction, camera enhancement, face/object detection, industrial anomaly detection, health wearables, autonomous robotics, offline translation, and privacy-preserving personalization.

12. Common Mistakes

- Optimizing on a desktop instead of the target device
- Counting only model file size, not peak RAM/runtime binary
- Using unsupported operators that silently fall back to CPU
- Excluding camera decoding/preprocessing from latency
- Ignoring sustained thermal throttling
- Randomly splitting correlated device/user data
- Shipping without signed updates and rollback
- Logging raw sensitive sensor data by default
- Assuming all devices of one OS have the same accelerator behavior

13. Edge Cases / Limitations

Device fragmentation creates inconsistent performance. Tiny models may fail on rare safety-critical cases. Sensor distributions drift with hardware aging or environment. Some accelerators require fixed shapes and specific quantization. Offline models cannot receive fresh server context. Physical attackers may extract model artifacts.

14. Variations

Variation	What changes / when to use	Importance
Mobile CPU/GPU/NPU	Rich OS and accelerator delegates	Placement/projects
Browser/WebGPU/WASM	No native install, sandboxed runtime	Web AI roles
Microcontroller/TinyML	Kilobytes/megabytes, integer kernels	Embedded research
Edge gateway	More power; aggregates local sensors	Industrial IoT
Hybrid edge-cloud	Local fast/private path plus cloud quality	Product systems
Federated learning	Local updates aggregated without raw-data centralization	Research/privacy

15. Related Topics

Quantization, structured pruning, and distillation create deployable small models. ONNX/TFLite/Core ML describe or execute artifacts. Caching avoids repeated sensor/model work. Custom kernels and hardware delegates accelerate hotspots. Federated learning concerns distributed training; edge deployment concerns inference location, though they often coexist.

16. Interview Questions

1. **Why deploy at the edge?** Lower network latency, privacy, offline operation, and bandwidth/cost savings.
2. **Main constraints?** Compute, peak RAM, storage, power, thermals, supported ops, and device fragmentation.
3. **Model size vs runtime memory?** File size is weights; runtime also needs activations, workspace, runtime, and buffers.

4. **Why benchmark on device?** Hardware delegates, memory, thermal limits, and runtimes differ from development machines.
5. **What is an accelerator delegate?** A backend that executes supported graph segments on GPU/NPU/DSP.
6. **What if one op is unsupported?** It may fall back to CPU and create costly device transfers; rewrite or keep graph consistently delegated.
7. **How do you handle model updates?** Versioned signed artifacts, staged rollout, compatibility checks, monitoring, and rollback.
8. **How do you reduce edge latency?** Smaller architecture/input, INT8, operator fusion, fewer conversions, efficient runtime, and target-specific profiling.
9. **What is hybrid inference?** Route suitable cases locally and harder/fresh-context cases to cloud.
10. **How do you evaluate energy?** Measure power over a sustained representative workload and integrate per inference.
11. **What privacy risks remain on edge?** Logs, outputs, stored features, model extraction, and insecure updates.
12. **How do you validate robustness?** Test real device/sensor subgroups, adverse conditions, offline behavior, and thermal duration.

17. Practice Tasks

- Export and benchmark a compact classifier in an edge runtime.
- Quantize it to INT8 and compare size/quality/on-device latency.
- Profile preprocessing versus inference time.
- Debug a graph split caused by one unsupported operator.
- Design an OTA schema with version, hash, signature, and rollback.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Offline plant diagnosis	Runs quantized vision on a phone	PyTorch, TFLite/ORT Mobile	PlantVillage + phone photos	Full edge lifecycle
TinyML sound detector	Detects events on MCU-class hardware	TensorFlow Lite Micro/CMSIS-NN	ESC-50 subset	Embedded AI depth
Hybrid privacy assistant	Local intent routing with cloud fallback	ONNX Runtime, mobile app, API	CLINC150	Product/system trade-offs

19. Quick Revision

- **Key idea:** run useful AI within real device resource and reliability limits.
- **Main constraint:** optimize quality subject to latency, RAM, energy, and size budgets.
- **When to use:** privacy, offline, instant response, or bandwidth matter.
- **Metrics:** subgroup quality, on-device p95, RAM, bytes, joules, thermals.
- **Common traps:** desktop-only tests, CPU fallback, ignoring preprocessing.
- **Interview one-liner:** Edge deployment is target-first co-design of model, runtime, hardware, and product fallback.

20. Final Cheat Sheet

Item	Summary
Definition	Inference near the data source on constrained devices
Input/output	Local sensor/user input -> local prediction/action
Main steps	Budget, compress, convert, validate, profile device, package/update
Hyperparameters	Input size, precision, threads, delegate, model width/depth
Metrics	Quality, p95, peak RAM, artifact size, energy, thermal stability
Pros	Privacy, offline, low network latency, bandwidth savings
Cons	Constraints, fragmentation, updates, limited model capacity
Best use cases	Real-time, private, offline, sensor-near applications

Custom Kernels

1. Overview

A kernel is a low-level function executed in parallel on an accelerator, such as matrix multiplication, normalization, attention, or a fused sequence of operations. Custom kernels are written or generated when existing framework/library kernels do not efficiently implement a measured hotspot. Common tools include CUDA C++, Triton language, PyTorch custom operators, and compiler systems such as [torch.compile](#).

Custom kernels can deliver major speedups but add correctness, numerical, portability, compilation, and maintenance risk. The best first step is usually graph/compiler fusion or a proven library kernel.

2. Intuition

A framework graph may send a worker to the warehouse separately for every ingredient. A fused kernel collects ingredients once, cooks several steps while data is close, and writes the final result once. The savings often come from less memory traffic and fewer launches, not fewer mathematical operations.

3. Prerequisites

Tensor shapes/strides, GPU execution model, threads/warps/blocks, memory hierarchy, coalescing, synchronization, numerical precision, profiling, and PyTorch autograd/operator integration.

4. Core Concepts

Subtopic	Meaning	Why it matters	Example	Common interview angle
Kernel launch	Dispatch GPU function	Overhead matters for tiny ops	Three elementwise launches	Why fuse?

Subtopic	Meaning	Why it matters	Example	Common interview angle
Thread/block mapping	Assign output work to threads	Controls parallelism	One thread per element	Grid calculation
Memory coalescing	Adjacent threads access adjacent addresses	Maximizes bandwidth	Contiguous vector load	Strides/layout
Shared memory	Fast block-local storage	Reuse and tiling	Matmul tile	Capacity/bank conflicts
Registers	Fast thread-local values	Avoid loads; can reduce occupancy	Accumulators	Register pressure
Occupancy	Active warps relative to limit	Hides latency, not sole goal	More resident blocks	Why 100% is not necessary
Fusion	Combine operators	Less traffic/launch overhead	Bias+GELU	Numerics/autograd
Tiling	Work on reusable subblocks	Improves locality	Blocked GEMM	Tile-size tuning
Autotuning	Benchmark configurations	Shape/hardware-specific choices	Block 128 vs 256	Compile/tuning cost

5. Algorithm / Working Process

1. Profile end-to-end and identify a stable dominant hotspot.
2. Confirm no existing optimized library/operator or compiler fusion handles it.
3. Specify exact semantics: shapes, strides, dtypes, devices, gradients, edge cases, and tolerances.
4. Map outputs to programs/threads and design memory access/tiling.
5. Implement the smallest forward kernel; add backward only if training requires it.
6. Test against a trusted reference over randomized shapes, dtypes, non-contiguous inputs if supported, and boundary sizes.
7. Benchmark with warm-up, proper synchronization, and representative shapes.
8. Integrate with dispatch/fallback and monitor across hardware/software versions.

6. Mathematical Foundation

For a fused bias and ReLU operation:

$$y_{ij} = \max(0, x_{ij} + b_j)$$

Separate execution reads/writes the intermediate $x+b$; fusion can read x, b and write y once. Arithmetic count is similar, but bytes moved fall.

The roofline bound is:

```
performance <= min(peak_FLOPs,
                    bandwidth * arithmetic_intensity)
arithmetic_intensity = FLOPs / bytes_moved
```

For a 1D tensor with N elements and block size B , grid size is:

```
number_of_programs = ceil(N / B)
offsets = program_id * B + [0, 1, ..., B-1]
mask = offsets < N
```

The mask prevents out-of-bounds access in the final partial block.

7. Practical Implementation

A Triton fused bias-ReLU kernel with a reference check:

```
import torch
import triton
import triton.language as tl

@triton.jit
def bias_relu_kernel(x_ptr, bias_ptr, out_ptr, n_elements, width: tl.constexpr,
                    BLOCK: tl.constexpr):
    offsets = tl.program_id(0) * BLOCK + tl.arange(0, BLOCK)
    mask = offsets < n_elements
    x = tl.load(x_ptr + offsets, mask=mask)
    bias = tl.load(bias_ptr + (offsets % width), mask=mask)
    tl.store(out_ptr + offsets, tl.maximum(x + bias, 0.0), mask=mask)

def bias_relu(x, bias):
    assert x.is_cuda and bias.is_cuda
    assert x.is_contiguous() and bias.is_contiguous()
    assert x.shape[-1] == bias.numel()
    out = torch.empty_like(x)
    n = x.numel()
    grid = (triton.cdiv(n, 256),)
    bias_relu_kernel[grid](x, bias, out, n, bias.numel(), BLOCK=256)
    return out

x = torch.randn(17, 513, device="cuda")
bias = torch.randn(513, device="cuda")
actual = bias_relu(x, bias)
expected = torch.relu(x + bias)
torch.testing.assert_close(actual, expected)
```

Before maintaining a kernel, test compiler fusion:

```
@torch.compile
def compiled_bias_relu(x, bias):
    return torch.relu(x + bias)
```

8. Code Explanation

Each Triton program processes **BLOCK** flattened elements. Modulo maps a flattened offset to the last-dimension bias index. Masked loads/stores handle a tensor whose element count is not divisible by 256. Assertions deliberately define the supported contract. The PyTorch expression is the correctness oracle. **torch.compile** may generate equivalent fusion with much lower maintenance, so it should be benchmarked first.

9. Training / Evaluation

For inference-only kernels, compare outputs and deployment metrics. For training, validate forward and backward against a reference using gradient checks where meaningful. Test random, zero, extreme, NaN/Inf policy, odd sizes, minimum/maximum shapes, and all supported dtypes/layouts. Benchmark p50 distribution with GPU events or a framework benchmark harness; report shape, dtype, device, warm-up, and compiler version.

10. Complexity and Cost

The fused elementwise example is $O(N)$ work and $O(N)$ memory traffic, but avoids an intermediate tensor and one launch. Kernel development cost can outweigh microseconds saved unless the operation dominates repeated execution. Compilation and autotuning add cold-start cost. Poor tile sizes, register pressure, or uncoalesced memory can make custom code slower than libraries.

11. Common Use Cases

Fused activation/norm operations, FlashAttention-style tiled attention, quantization/dequantization, sparse kernels, specialized reductions, MoE routing, image preprocessing, and domain-specific scientific operators.

12. Common Mistakes

- Writing a kernel before profiling
- Reimplementing cuBLAS/cuDNN/FlashAttention with no demonstrated gap
- Timing asynchronous launches without synchronization
- Testing only dimensions divisible by block size
- Assuming contiguous layout without enforcing it
- Ignoring FP16 accumulation/overflow behavior
- Missing backward/autograd integration for training
- Benchmarking one friendly shape and claiming general speedup
- Increasing occupancy while reducing actual performance
- No safe fallback for unsupported shapes/dtypes/devices

13. Edge Cases / Limitations

Non-contiguous tensors, broadcasting, empty dimensions, huge indexing ranges, odd tails, NaN semantics, mixed precision, deterministic reductions, and device differences require explicit handling. Atomics can be nondeterministic. A kernel tuned for one GPU/shape may regress on another. Compiler/runtime updates can outperform or break assumptions.

14. Variations

Variation	What changes / when to use	Importance
<code>torch.compile</code> fusion	Compiler generates kernels from Python graph	First practical choice
Triton	Python-like GPU kernel DSL	Strong AI systems skill
CUDA C++ extension	Maximum low-level control	GPU engineer roles
Custom ONNX/TensorRT op	Runtime-specific integration/plugin	Deployment roles
CPU SIMD kernel	AVX/NEON vectorization	CPU/edge optimization
Autograd custom op	Defines forward and backward	Training workloads

15. Related Topics

TensorRT selects and fuses kernels automatically; plugins introduce custom ones. Quantization and sparsity need dtype/pattern-specific kernels to become fast. Batch size changes kernel shapes and utilization. FlashAttention is an IO-aware exact attention algorithm/kernel, not merely an approximation. ONNX custom operators carry semantic and portability costs.

16. Interview Questions

1. **What is a GPU kernel?** A function executed in parallel by many GPU threads over data.
2. **When should you write one?** After profiling shows a material hotspot not handled by libraries/compilers.
3. **Why fuse operations?** Reduce intermediate memory traffic and kernel-launch overhead.
4. **What is coalesced access?** Adjacent threads access adjacent memory locations, enabling efficient transactions.
5. **What is tiling?** Partition work into blocks to reuse data in registers/shared memory.
6. **What is occupancy?** Fraction of hardware's possible resident warps; it helps hide latency but does not equal performance.
7. **What is arithmetic intensity?** FLOPs performed per byte moved.
8. **Compute-bound vs memory-bound?** Limited by arithmetic throughput versus data bandwidth; optimize accordingly.
9. **Why mask the last block?** Grid sizes round up, so some lanes otherwise access beyond tensor bounds.
10. **How do you benchmark kernels?** Warm up, synchronize/use device events, repeat, and compare representative shapes to a correct baseline.
11. **Why can a custom kernel be slower?** Bad access, launch config, register pressure, small workload, compilation, or superior vendor kernels.
12. **What is the first alternative to custom code?** A proven library op or compiler fusion such as `torch.compile`.

17. Practice Tasks

- Benchmark `torch.compile` fusion for bias+activation.
- Implement a masked Triton vector operation for odd sizes.
- Use a profiler to separate launch, compute, and memory bottlenecks.
- Debug incorrect outputs from missing boundary masks.
- Extend a forward kernel with a tested backward implementation.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Fusion benchmark suite	Compares eager, compiled, and Triton ops	PyTorch, Triton, profiler	Synthetic tensors	Kernel measurement rigor
Fused normalization op	Implements/tests RMSNorm variants	Triton/CUDA, PyTorch	Transformer shapes	LLM performance relevance
Sparse/quantized kernel	Accelerates a chosen supported pattern	Triton/CUDA	Synthetic + model trace	Advanced systems portfolio

19. Quick Revision

- **Key idea:** specialize parallel execution and reduce data movement for a proven hotspot.
- **Main model:** roofline $\min(\text{peak compute}, \text{bandwidth} * \text{intensity})$.
- **When to use:** libraries/compiler cannot meet a measured requirement.
- **Metrics:** correctness, kernel/end-to-end latency, bandwidth, occupancy, compile time.
- **Common traps:** premature kernels, async timing, missing tails/layouts.
- **Interview one-liner:** Custom kernels earn their maintenance cost only when profiling, correctness tests, and end-to-end benchmarks all agree.

20. Final Cheat Sheet

Item	Summary
Definition	Specialized low-level parallel accelerator/CPU operation
Input/output	Tensors with explicit contract -> computed tensors
Main steps	Profile, specify, map/tile, implement, verify, benchmark, fallback
Hyperparameters	Block/tile size, warps, stages, dtype, layout
Metrics	Correctness tolerance, latency, bandwidth, occupancy, speedup
Pros	Fusion, locality, shape-specific performance
Cons	Complexity, portability, numerical and maintenance risk
Best use cases	Stable high-impact hotspots unmet by existing kernels

Cross-Topic Deployment Checklist

1. Establish a reproducible quality and performance baseline.
2. Profile before choosing compression: compute-bound, memory-bound, queue-bound, or network-bound.
3. Choose the least disruptive technique that attacks the measured bottleneck.
4. Validate task quality after every conversion or precision change.
5. Benchmark on target hardware with representative shapes, warm-up, synchronization, and latency percentiles.
6. Report end-to-end metrics alongside isolated model/kernel metrics.
7. Version model, tokenizer/preprocessing, runtime, precision, cache namespace, and hardware profile.
8. Test fallbacks, invalid shapes, partial batches, cache invalidation, and resource exhaustion.
9. Monitor quality drift, p95/p99, throughput/goodput, memory, errors, and cost after deployment.
10. Keep the original model and a rollback path until the optimized artifact proves stable.